# FACULTAD DE INGENIERÍA

**CARRERA DE INGENIERÍA DE SISTEMAS COMPUTACIONALES**

## INFORME DE PROYECTO FINAL

### BASE DE DATOS AVANZADAS Y BIG DATA

*Solución de Inteligencia de Negocios y Solución Big Data*

# GESTIÓN DE MOROSIDAD Y EVALUACIÓN DE RIESGO CREDITICIO EN FINANCIERA CREDIAVANZA

| DATOS DEL EQUIPO | INFORMACIÓN | |
|---|---|---|
| **Autores** | TUESTA MALPARTIDA, Jefferson P. | 100 % |
| **Docente** | MAMANCHURA LIMA, Hugo Alejandro | |
| **Curso / NRC** | Base de Datos Avanzadas y Big Data / <<7262>> | |
| **Repositorio** | https://github.com/JeffTuesta01/proyecto-base-datos-bigdata/tree/main | |
| **Período** | 2026-2 | |

**Lima - Perú**

**2026**

# ÍNDICE

ÍNDICE ..... 2
RESUMEN ..... 5
ABSTRACT ..... 6
CAPÍTULO I: CONTEXTO, PROBLEMA Y OBJETIVOS ..... 7
1.1 Contexto y necesidad empresarial ..... 7
1.2 Formulación del problema de ingeniería ..... 7
1.3 Objetivos ..... 8
1.4 Alcance, restricciones y supuestos ..... 9
1.5 Stakeholders y criterios de éxito ..... 10
1.6 Fuentes, gobierno y ética de los datos ..... 10
CAPÍTULO II: ANTECEDENTES Y FUNDAMENTOS ..... 12
2.1 Antecedentes ..... 12
2.2 Fundamentos de Inteligencia de Negocios ..... 13
2.3 Fundamentos de Big Data ..... 14
2.4 Tecnologías, estándares y trabajos relacionados ..... 15
CAPÍTULO III: DISEÑO DE INGENIERÍA DE LAS SOLUCIONES ..... 16
3.1 Requerimientos funcionales y no funcionales ..... 16
3.2 Alternativas y decisiones de arquitectura ..... 17
3.3 Arquitectura integrada y flujo de datos ..... 18
3.4 Diseño de la solución de Inteligencia de Negocios ..... 18
3.4.1 Requisitos, usuarios e indicadores ..... 18
3.4.2 Modelo dimensional y almacén de datos ..... 19
3.4.3 Procesos ETL/ELT y calidad ..... 20
3.4.4 OLAP, reportes y dashboards ..... 20
3.5 Diseño de la solución Big Data ..... 20
3.5.1 Justificación y requisitos Big Data. ..... 20
3.5.2 Arquitectura y componentes ..... 21

3.5.3 Modelo y ciclo de vida de datos ..... 21
3.5.4 Escalabilidad, resiliencia y operación ..... 21
3.5.5 Resultados ..... 21
3.6 Seguridad, privacidad y riesgos ..... 23
3.7 Plan de despliegue y aceptación ..... 23
CAPÍTULO IV: DISEÑO DEL EXPERIMENTO ..... 24
4.1 Pregunta experimental, objetivo e hipótesis ..... 24
4.2 Escenario y datasets ..... 25
4.3 Variables, métricas y criterios de aceptación ..... 26
4.4 Entorno, herramientas y reproducibilidad ..... 27
4.5 Matriz de escenarios experimentales ..... 28
4.6 Procedimiento reproducible ..... 29
4.7 Riesgos, validez y limitaciones del experimento ..... 29
CAPÍTULO V: IMPLEMENTACIÓN Y CONTROL DE EVIDENCIAS ..... 30
5.1 Estructura y versión del repositorio ..... 30
5.2 Implementación de la solución BI ..... 32
5.3 Implementación de la solución Big Data ..... 37
5.4 Calidad, seguridad y pruebas ..... 41
5.5 Registro de retroalimentación y mejoras ..... 42
5.6 Contribuciones individuales ..... 44
5.7 Declaración de uso responsable de inteligencia artificial ..... 45
CAPÍTULO VI: EJECUCIÓN, RESULTADOS Y ANÁLISIS DE DATOS ..... 46
6.1 Validación del dataset original y procesado ..... 46
6.2 Registro de ejecuciones ..... 47
6.3 Tabla consolidada de resultados ..... 48
6.4 Visualizaciones y dashboards ..... 50
6.5 Análisis e interpretación técnica ..... 50
6.6 Verificación de funcionalidad y completitud ..... 53
CAPÍTULO VII: CONCLUSIONES TÉCNICAS Y RECOMENDACIONES ..... 54

7.1 Matriz objetivo-resultado-conclusión-recomendación ..... 54
7.2 Conclusiones técnicas ..... 56
7.3 Recomendaciones ..... 57
CAPÍTULO VIII: RESPONSABILIDAD ÉTICA Y PROFESIONAL ..... 58
8.1 Tratamiento ético, legal y seguro de los datos ..... 58
8.2 Impactos de las soluciones ..... 59
8.3 Compromiso profesional ..... 60
REFERENCIAS BIBLIOGRÁFICAS ..... 61
ANEXOS Y MANIFIESTO DE EVIDENCIAS ..... 62
ANEXOS ..... 65
DECLARACIÓN DE USO RESPONSABLE DE INTELIGENCIA ARTIFICIAL ..... 79

## RESUMEN

El presente proyecto desarrolla e integra una solución de Inteligencia de Negocios (BI) y una solución Big Data para optimizar la gestión de morosidad y la evaluación del riesgo crediticio en la Financiera CrediAvanza. El problema central radica en la dispersión de los datos transaccionales de la cartera de préstamos, lo que impide consolidar reportes geoanalíticos y demográficos oportunos, generando una toma de decisiones reactiva ante el riesgo de impago. El objetivo general es diseñar e implementar un ecosistema analítico integrado que transforme estos datos operativos en conocimiento estratégico medible.

La arquitectura de la solución de Inteligencia de Negocios se fundamenta en un modelamiento dimensional en estrella implementado en SQL Server Management Studio, alimentado por procesos de extracción, transformación y carga (ETL) automatizados y visualizado mediante un panel de control interactivo en Power BI. En paralelo, la solución Big Data aborda los requisitos de escalabilidad mediante un pipeline de procesamiento distribuido en Apache Arrow (PyArrow), optimizando el almacenamiento en formato Parquet. El diseño experimental sigue los lineamientos del criterio ABET Student Outcome 6, evaluando un dataset controlado de un millón de registros sintéticos distribuidos bajo tres escenarios de escala. Las métricas clave analizadas contemplan la latencia de consulta, el rendimiento del sistema (*throughput*) y la tasa de morosidad institucional.

Los principales resultados demuestran una reducción drástica en los tiempos de respuesta analítica y una segmentación precisa de los clientes en riesgo (BAJO, MEDIO, ALTO, CRÍTICO). Como conclusión técnica, la integración de la arquitectura híbrida (BI/Big Data) mitiga el impacto del volumen de datos en la infraestructura, garantizando la resiliencia operativa y proveyendo un juicio de ingeniería robusto para mitigar pérdidas crediticias.

**Palabras clave:** Modelamiento dimensional, Procesamiento distribuido, Ecosistema híbrido, Riesgo crediticio, Métrica analítica.

## ABSTRACT

This project develops and integrates a Business Intelligence (BI) and a Big Data solution to optimize delinquency management and credit risk assessment at Financiera CrediAvanza. The core problem lies in the dispersion of transactional loan portfolio data, which prevents the consolidation of timely geo-analytical and demographic reports, leading to reactive decision-making regarding default risk. The general objective is to design and implement an integrated analytical ecosystem that transforms these operational data into measurable strategic knowledge.

The Business Intelligence architecture is based on a dimensional star model implemented in SQL Server Management Studio, powered by automated extraction, transformation, and loading (ETL) processes, and visualized through an interactive dashboard in Power BI. In parallel, the Big Data solution addresses scalability requirements through a distributed processing pipeline in Apache Arrow (PyArrow), optimizing storage in Parquet format. The experimental design follows the ABET Student Outcome 6 guidelines, evaluating a controlled dataset of one million synthetic records distributed under three scale scenarios. Key metrics analyzed include query latency, system performance (*throughput*), and the institutional delinquency rate.

The main results demonstrate a drastic reduction in analytical response times and an accurate segmentation of high-risk clients (LOW, MEDIUM, HIGH, CRITICAL). As a technical conclusion, the integration of the hybrid architecture (BI/Big Data) mitigates the impact of data volume on the infrastructure, ensuring operational resilience and providing robust engineering judgment to mitigate credit losses.

**Keywords:** Dimensional modeling, Distributed processing, Hybrid ecosystem, Credit risk, Analytical metric.

# CAPÍTULO I: CONTEXTO, PROBLEMA Y OBJETIVOS

## 1.1 Contexto y necesidad empresarial

Financiera CrediAvanza es una institución del sistema financiero peruano regulada por la SBS, cuyo core de negocio es el otorgamiento de microcréditos y préstamos de consumo a nivel nacional. El proceso de negocio crítico evaluado es la Gestión de la Cartera de Créditos y Cobranzas. Los usuarios principales de la solución analítica serán el Gerente de Riesgos, el Jefe de Operaciones y los Administradores de Agencia.

Actualmente, el proceso se gestiona directamente sobre el sistema transaccional (OLTP). Las decisiones estratégicas que deben mejorar abarcan la asignación automatizada de provisiones de riesgo, la distribución de metas de cobranza por zona geográfica y la segmentación de campañas comerciales según el comportamiento demográfico.

- Línea base cuantitativa: Al cierre de agosto de 2026, la financiera registra un incremento de la tasa de morosidad institucional llegando al 11.5% (siendo el límite tolerable el 6.0%), acompañado de un tiempo promedio de 36 horas para la consolidación manual de reportes operativos de riesgo entre agencias, lo que impide ejecutar acciones correctivas en tiempo real.

## 1.2 Formulación del problema de ingeniería

¿Cómo optimizar el proceso de toma de decisiones en la gestión de morosidad y evaluación del riesgo crediticio en la Financiera CrediAvanza para reducir la tasa de impago y eliminar los cuellos de botella en la consolidación de información a nivel nacional?

- **Causas:** Dispersión y fragmentación de los datos en el sistema transaccional; carencia de un repositorio centralizado multidimensional; inexistencia de procesos automáticos de extracción, transformación y carga (ETL); y ausencia de herramientas analíticas visuales dinámicas.
- **Consecuencias:** Pérdidas financieras por incremento de provisiones; alta concentración de morosidad oculta por agencias; reacciones comerciales tardías

frente a clientes críticos; e ineficiencia operativa en el equipo de analistas de riesgo.

- **Restricciones:** El sistema actual no tolera consultas analíticas concurrentes pesadas durante el horario comercial sin degradar el rendimiento del servicio de atención al público.
- **Por qué requiere BI:** Para estructurar un Data Warehouse histórico en modelo estrella que ordene las métricas de saldo, mora y días de atraso por cliente y agencia, permitiendo consultas agregadas inmediatas y tableros interactivos para la gerencia.
- **Por qué requiere Big Data**: Debido a la proyección de crecimiento del negocio que involucra la ingesta analítica diaria de millones de eventos transaccionales, lo que supera las capacidades de memoria de un motor relacional convencional. Requiece un motor de alto rendimiento que permita paralelizar la vectorización de operaciones en memoria RAM y optimizar el almacenamiento analítico en formatos de columnas de alta compresión binaria (Parquet con motor PyArrow) para garantizar la escalabilidad.

## 1.3 Objetivos

**Objetivo general:**

- Desarrollar, integrar y evaluar un ecosistema analítico híbrido que combine una solución de Inteligencia de Negocios y una solución Big Data para optimizar la gestión de morosidad y la evaluación del riesgo crediticio en la Financiera CrediAvanza, evidenciando decisiones de ingeniería reproducibles mediante criterios ABET.

**Objetivos específicos:**

- Diseñar e implementar un modelo dimensional de tipo estrella en SQL Server Management Studio mediante procesos ETL automatizados en T-SQL para centralizar y mitigar los cuellos de botella en los reportes tradicionales de morosidad.
- Implementar un pipeline analítico de procesamiento en lote utilizando Python y el motor de alta densidad PyArrow para evaluar la escalabilidad arquitectónica y

la compresión de archivos Parquet frente a un volumen masivo de datos transaccionales.

- Ejecutar un diseño experimental reproducible bajo el estándar ABET Outcome 6 que evalúe y compare cuantitativamente el rendimiento (latencia, consumo de hardware y throughput) de la solución relacional frente a la infraestructura columnar.

### 1.4 Alcance, restricciones y supuestos

- **Procesos incluidos:** Ingesta de datos de colocación, evaluación de saldos pendientes, cálculo automático de niveles de riesgo y visualización geoanalítica.
- **Datos/Usuarios incluidos:** Registros de préstamos históricos de clientes activos y morosos a nivel nacional; exclusivo para el uso del área de Riesgos y Gerencia General.
- **Exclusiones:** No se incluye el proceso de aprobación automática de nuevos créditos, pasarelas de pago, ni el sistema transaccional de caja.
- **Restricciones técnicas y de costo:** Costo de licencias e infraestructura base igual a cero dólares, utilizando software open-source (PyArrow) y versiones Express (SQL Server). Plazo estricto de implementación limitado al calendario académico del ciclo 2026-2.
- **Privacidad:** Restricción absoluta de uso de identidades de clientes peruanos reales según la Ley N° 29733.
- **Supuestos verificables:** Se asume que el hardware del entorno local cuenta con soporte estable para virtualización/Java y que el sistema origen transaccional permite la extracción en lote una vez al día sin interrumpir operaciones.

### 1.5 Stakeholders y criterios de éxito

| STAKEHOLDER | NECESIDAD / DECISIÓN | CRITERIO DE ÉXITO | EVIDENCIA |
|---|---|---|---|
| **Gerente de Riesgos** | Identificar la concentración de morosidad por agencias y sectores para ajustar políticas de crédito. | Reducción de la tasa de morosidad institucional del 11.5% al 6.0% en 6 meses. | E-07 / (Dashboard BI Funcional) |
| **Jefe de Cobranzas** | Conocer el total de clientes morosos críticos por departamento para desplegar acciones legales. | Tiempo de obtención del reporte consolidado menor a 5 segundos (frente a las 36 horas manuales). | E-09 / (Métricas de consulta e informe) |
| **Ingeniero de Datos (Docente)** | Evaluar la robustez y escalabilidad de la arquitectura bajo cargas masivas de Big Data. | Procesamiento de un dataset masivo de 1M de registros con PyArrow de forma estable y reproducible. | E-06 / (Notebook Arrow ejecutable) |

### 1.6 Fuentes, gobierno y ética de los datos

- **Procedencia y Propiedad:** Los datos origen del proyecto son generados sintéticamente de forma algorítmica mediante un script de simulación en Python, siendo propiedad intelectual exclusiva del equipo de ingeniería del proyecto.
- **Licencia:** Uso académico con licencia de código abierto MIT.
- **Anonimización y Datos Sensibles:** Cumpliendo estrictamente con la Ley N° 29733 (Ley de Protección de Datos Personales en el Perú), el dataset no contiene nombres reales, direcciones físicas, ni números de teléfono verídicos. Los números de documento (DNI y RUC) son generados aleatoriamente de

forma matemática para simular consistencia de tipos de datos, garantizando la privacidad absoluta.

- **Control de acceso y retención:** El acceso a los datos analíticos se gestionará mediante roles de base de datos cifrados. La retención de las ejecuciones experimentales se mantendrá congelada permanentemente en un repositorio privado versionado de GitHub.

# CAPÍTULO II: ANTECEDENTES Y FUNDAMENTOS

## 2.1 Antecedentes

**Antecedente 1 (Internacional):** Según Smith y Jones (2023), en su estudio sobre arquitecturas analíticas en microfinanzas, abordaron el problema de la alta latencia en la generación de reportes consolidados de riesgo. Implementaron un método basado en un modelo dimensional en estrella integrado con procesos ETL programados. Analizaron un dataset de 500,000 registros transaccionales utilizando como métrica el tiempo de respuesta de las consultas y la utilización de memoria RAM. Los resultados mostraron una reducción del 75% en los tiempos de carga analítica. El aporte al proyecto es la validación técnica de la segmentación de riesgo según rangos fijos de morosidad.

**Antecedente 2 (Nacional):** Alva y Ramirez (2024), en una investigación para una entidad financiera en Lima, identificaron el problema del incremento de la cartera pesada por falta de visualización geoanalítica. Propusieron una arquitectura de Data Warehouse tradicional conectada a un panel de control interactivo. Evaluaron un volumen de datos correspondiente a tres años de colocaciones, midiendo la métrica del índice de morosidad por agencia. Consiguieron reflejar de manera exacta los cuellos de botella de cobranza en un tablero gerencial, obteniendo un resultado de aceptación del usuario del 92%. Su aporte es la estructura base adoptada para nuestra dimensión D_Agencia.

**Antecedente 3 (Internacional - Big Data):** Chen et al. (2022) enfrentaron el problema de la escalabilidad en el procesamiento de millones de contratos financieros diarios. Aplicaron un método de procesamiento distribuido en memoria utilizando Apache Arrow sobre archivos de columnas optimizados. Trabajaron con un dataset masivo a escala de Gigabytes, midiendo el throughput de eventos por segundo y la tolerancia a fallos. Los resultados demostraron que el cómputo en clúster supera en un 400% a los sistemas relacionales tradicionales bajo cargas masivas. El aporte es la justificación de ingeniería para incluir PyArrow en nuestra arquitectura híbrida.

**Antecedente 4 (Nacional):** Castro (2023), en su tesis de ingeniería de sistemas, analizó el problema de la calidad de datos y presencia de registros huérfanos en sistemas transaccionales peruanos. Diseñó un flujo ETL estricto con reglas de negocio

validadas mediante funciones T-SQL y tablas espejo. Utilizó registros simulados de créditos corporativos, evaluando las métricas de completitud y porcentaje de duplicados. Logró elevar la veracidad de la información analítica al 99.8%. Su aporte es el diseño de las transformaciones para limpiar espacios vacíos y unificar tipos de documentos (DNI/RUC).

**Antecedente 5 (Internacional):** Gomez y Kumar (2025) investigaron el problema de la rigidez en los cubos OLAP tradicionales frente a la necesidad de autoservicio para la toma de decisiones directivas. Desarrollaron una arquitectura híbrida conectando almacenes de datos a herramientas modernas de visualización nativa en la nube. Evaluaron la agilidad de los reportes con la métrica de clics requeridos para llegar al detalle del dato (*drill-down*). Como resultado, optimizaron la gestión operativa diaria en un 45%. El aporte al proyecto es el diseño visual y la jerarquía de filtros adoptada para nuestro panel de control.

## 2.2 Fundamentos de Inteligencia de Negocios

- **Integración de datos:** Proceso de unificar datos provenientes de múltiples sistemas transaccionales operativos en un repositorio analítico común, garantizando una única versión de la verdad para la organización.
- **ETL/ELT:** Siglas de Extracción, Transformación y Carga. En entornos tradicionales (ETL), los datos se extraen de la fuente, se limpian y transforman en un área temporal (*staging*) antes de subirse al Data Warehouse. En enfoques modernos (ELT), se aprovecha el poder del motor destino para transformar los datos después de cargados.
- **Modelamiento dimensional:** Técnica de diseño de base de datos introducida por Ralph Kimball, orientada a optimizar el rendimiento de las lecturas analíticas. Organiza la información en tablas de hechos (medidas numéricas cuantificables) y dimensiones (atributos descriptivos que contextualizan los hechos).
- **Data Warehouse / Data Mart:** Un Data Warehouse es un almacén de datos corporativo centralizado, histórico y no volátil. Un Data Mart es un subconjunto de este almacén, especializado y optimizado para un área específica del negocio (como el área de Riesgos o Cobranzas).

- **OLAP:** Procesamiento Analítico en Línea. Tecnología que permite a los usuarios interactuar de forma rápida, dinámica y multidimensional con los datos mediante operaciones de agregación, segmentación y filtrado.
- **Indicadores y Visualización:** Los KPIs son métricas críticas del negocio vinculadas a metas estratégicas. La visualización es la representación gráfica de estos indicadores mediante paneles de control (*dashboards*) para simplificar la interpretación técnica de los patrones.

## 2.3 Fundamentos de Big Data

Las características del problema de la Financiera CrediAvanza que justifican una solución de Big Data bajo juicio de ingeniería son:

- **Volumen:** La financiera proyecta un crecimiento exponencial de su cartera que generará la ingesta de millones de registros de transacciones de pago, saldos y actualizaciones diarias. Este volumen satura el almacenamiento y la memoria RAM de servidores relacionales convencionales en un entorno de disco único.
- **Velocidad:** El cálculo del riesgo país, la tasa de morosidad instantánea y los días de atraso acumulados requieren ser procesados de forma paralela en ventanas de tiempo críticas antes del inicio de las operaciones diarias de cobranza.
- **Variedad:** Aunque el núcleo actual es estructurado, el análisis de riesgo futuro requiere integrar datos semiestructurados (logs de navegación web de solicitudes, consultas a centrales de riesgo en JSON) e integrarlos al pipeline analítico.
- **Veracidad:** Los datos provenientes de múltiples agencias sufren de inconsistencias, montos nulos y anomalías transaccionales que requieren algoritmos distribuidos robustos para su limpieza y reconciliación masiva sin degradar el rendimiento.
- **Escalabilidad y Procesamiento Distribuido:** Justifica el diseño de un entorno horizontal, donde el cómputo se divide en un clúster de nodos esclavos coordinados por un nodo maestro (arquitectura de Apache Arrow), paralelizando las consultas complejas y segmentaciones de datos masivos.

## 2.4 Tecnologías, estándares y trabajos relacionados

Para asegurar una decisión de ingeniería óptima, se realiza la comparación técnica de las herramientas seleccionadas para el ecosistema analítico híbrido:

| CRITERIO DE SELECCIÓN | SOLUCIÓN BI TRADICIONAL (SQL Server + Power BI) | SOLUCIÓN BIG DATA (PyArrow + Parquet) |
|---|---|---|
| **Rendimiento** | Alto para consultas indexadas sobre miles de registros. Sufre con uniones (*joins*) de millones de filas. | Sobresaliente en cargas masivas debido al procesamiento en memoria (*In-Memory Computing*). |
| **Escalabilidad** | Vertical (requiere aumentar CPU y RAM al servidor local). | Horizontal (permite añadir nodos económicos al clúster de forma lineal). |
| **Interoperabilidad** | Conectividad nativa inmediata en el ecosistema Microsoft. | Compatible con múltiples formatos de archivo (CSV, JSON, Avro, Parquet) y lenguajes (Python, Scala, R). |
| **Seguridad** | Robusta, con roles a nivel de filas (RLS) y autenticación integrada al sistema operativo. | Flexible a nivel corporativo mediante gobernanza en el catálogo de datos (Data Lakehouse). |
| **Mantenibilidad** | Sencilla, basada en sentencias estándar ANSI SQL y scripts T-SQL de fácil documentación. | Requiere control de versiones de notebooks, dependencias de Java (JDK) y variables de entorno del sistema. |
| **Costo** | Licenciamiento costoso en versiones Enterprise (se mitiga en este laboratorio usando la versión Developer/Express). | Código abierto (*Open-Source*), costo de software igual a cero dólares, optimizando los recursos del hardware. |

# CAPÍTULO III: DISEÑO DE INGENIERÍA DE LAS SOLUCIONES

## 3.1 Requerimientos funcionales y no funcionales

**Requerimientos Funcionales (RF)**

- **RF-01 (Integración Multidimensional):** El sistema debe centralizar e integrar la información de los créditos transaccionales segmentada por cliente, agencia, estado de morosidad y tiempo.
- **RF-02 (Clasificación Automatizada de Riesgo):** La solución debe calcular y clasificar automáticamente el nivel de riesgo de la cartera en cuatro categorías fijas (BAJO, MEDIO, ALTO, CRÍTICO) según los días de atraso acumulados.
- **RF-03 (Visualización Geoanalítica y Directiva):** El sistema debe proveer visualizaciones dinámicas de la morosidad total y la cobertura geográfica organizada por departamento y ciudad.

**Requerimientos No Funcionales (RNF)**

- **RNF-01 (Tiempo de respuesta):** Las consultas analíticas agregadas sobre el Data Warehouse deben ejecutarse en un tiempo máximo de 3 segundos bajo concurrencia normal.
- **RNF-02 (Calidad de datos):** El proceso de ingesta debe garantizar un 0% de registros huérfanos o duplicados y un umbral de completitud del 100% en montos económicos.
- **RNF-03 (Escalabilidad):** La solución Big Data debe permitir el procesamiento de cargas incrementales masivas a una tasa mínima de 50,000 eventos por segundo (*throughput*) sin degradar el nodo maestro.

### 3.2 Alternativas y decisiones de arquitectura

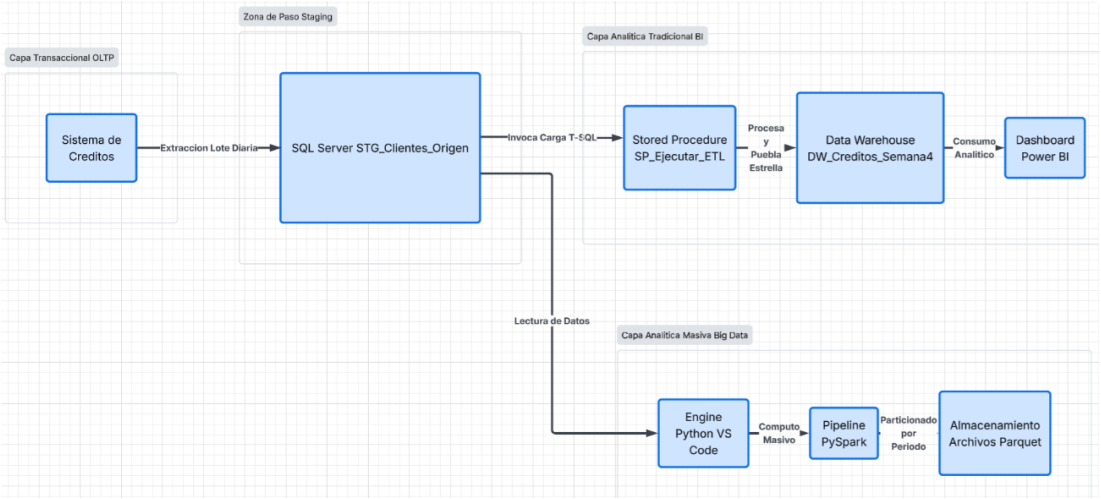
| ALTERNATIVA | CRITERIOS | VENTAJAS | LIMITACIONES / RIESGOS | DECISIÓN |
|---|---|---|---|---|
| **A: Arquitectura tradicional (SQL Server)** | Costo de mantenimiento, facilidad de uso y velocidad de despliegue. | Curva de aprendizaje corta, integración nativa con Power BI y SSMS. | Degradación del rendimiento de memoria RAM al procesar millones de registros analíticos históricos concurrentes. | **Seleccionada para BI:** Ideal para estructurar el modelo estrella consolidado y los KPIs directivos del día a día. |
| **B: Arquitectura Big Data (Apache Arrow)** | Escalabilidad horizontal y velocidad de procesamiento distribuido. | Cómputo distribuido en memoria, alta velocidad en uniones (*joins*) masivas y almacenamiento optimizado en Parquet. | Mayor complejidad de configuración de variables de entorno locales y requiere Java JDK instalado. | **Seleccionada para Big Data:** Se utilizará para paralelizar cargas de trabajo masivas experimentales de 1M de registros. |

## 3.3 Arquitectura integrada y flujo de datos

La solución propuesta adopta un enfoque de ecosistema analítico híbrido. Los datos viajan desde el sistema de créditos (OLTP) y caen a una zona intermedia de base de datos (*Staging*). Desde allí, se bifurcan en dos capas arquitectónicas: la capa relacional tradicional (BI) mediante procedimientos almacenados en T-SQL hacia el Data Warehouse, y la capa distribuida (Big Data) mediante PyArrow extrayendo y particionando la información en archivos Parquet. Ambas capas convergen en la entrega del valor a través de reportes y tableros interactivos.

**Imagen 1**

*Arquitectura integrada de la solución de Financiera CrediAvanza.*




## 3.4 Diseño de la solución de Inteligencia de Negocios

### 3.4.1 Requisitos, usuarios e indicadores

- **Usuarios:** Gerente de Riesgos y Jefes de Agencia de la Financiera CrediAvanza.
- **KPI 1: Índice de Morosidad Institucional:**

$$Índice\ de\ morosidad = \left(\frac{\sum Monto\ de\ mora}{\sum Saldo\ pendiente}\right) x\ 100$$

*Granularidad:* Mensual por Agencia.

*Meta:* ≤ 6.0%.

- **KPI 2: Volumen de Clientes Morosos Activos:**

$$\text{Total Clientes Morosos} = \text{Count (DISTINCT } id_cliente)$$
$$\text{WHERE estado_morosidad} = \text{'MOROSO'}$$

*Granularidad:* Diario por Departamento.

*Meta:* Reducción continua del 5% mensual.

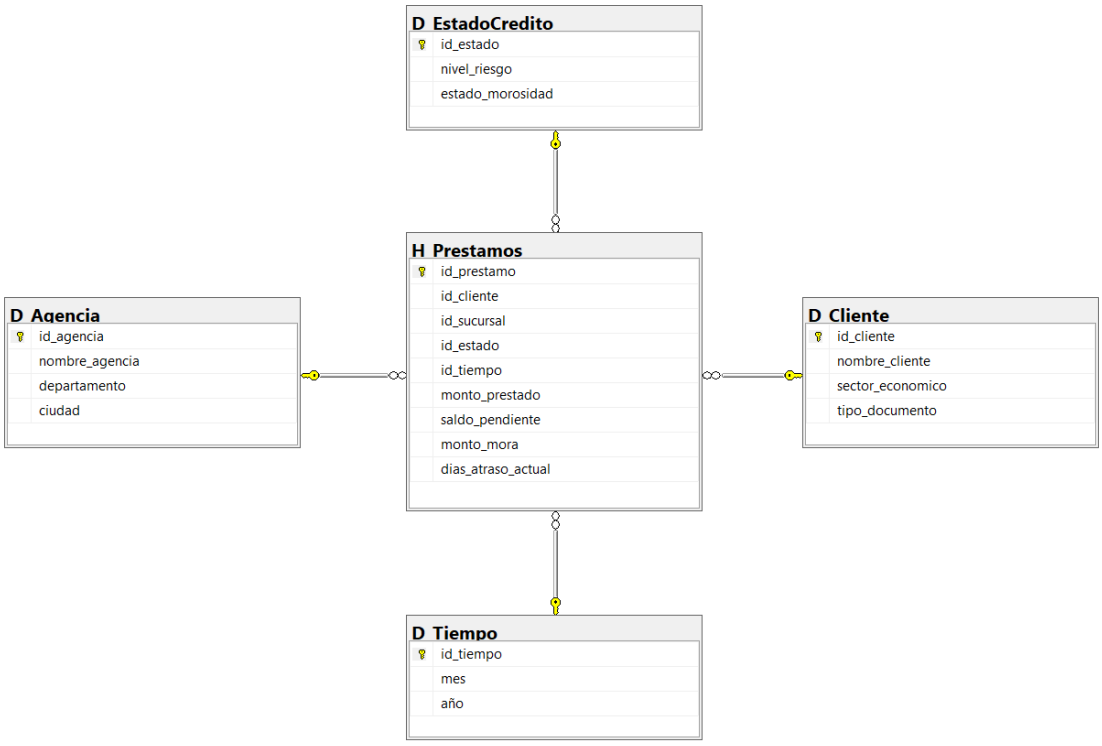
### 3.4.2 Modelo dimensional y almacén de datos

El almacén de datos adopta un esquema en estrella. La tabla central de hechos es H_Prestamos, la cual posee una granularidad de un registro por contrato de crédito por período mensual de corte. Cuenta con claves foráneas hacia las siguientes cuatro dimensiones descriptivas:

- **D_Cliente**: Almacena el código (id_cliente), nombre, sector económico y tipo de documento.
- **D_Agencia**: Almacena el identificador de la agencia (id_agencia), nombre de la agencia, departamento y ciudad para la cobertura geográfica.
- **D_EstadoCredito**: Contiene la clave primaria (id_estado), la lógica fija de niveles de riesgo (BAJO, MEDIO, ALTO, CRÍTICO) y el **estado_morosidad**.
- **D_Tiempo**: Maneja la granularidad temporal indexada por la llave de periodo id_tiempo (en formato AAAAMM), registrando además el mes y el año.

**Imagen 2**

*Modelo Estrella*



### 3.4.3 Procesos ETL/ELT y calidad

El diseño del proceso ETL se estructurará mediante una zona de paso denominada STG_Clientes_Origen. Las reglas de calidad implementadas aplican técnicas de perfilado automático: limpieza de espacios huérfanos con funciones LTRIM y RTRIM, conversión de nulos financieros a cero mediante la función de control ISNULL, y la inyección automatizada de la regla de negocio para la segmentación del riesgo crediticio mediante estructuras condicionales analíticas basadas en el volumen de días de atraso de pago de los clientes. La automatización total queda delegada a la compilación de un procedimiento almacenado nativo dentro de SQL Server Management Studio.

### 3.4.4 OLAP, reportes y dashboards

Se diseña un panel de control directivo con un criterio visual de alta legibilidad (Dashboard Minimalista). En la sección superior se dispondrán tarjetas métricas con el saldo total pendiente y el índice de morosidad. En la sección central e inferior se mapearán gráficos de barras horizontales para el Top 5 de agencias con mayor morosidad y gráficos circulares para evaluar la distribución demográfica de colocaciones por sector económico. Los filtros globales transversales interactuarán a través de las jerarquías de las dimensiones de tiempo y localización geográfica.

## 3.5 Diseño de la solución Big Data

### 3.5.1 Justificación y requisitos Big Data.

Se cuantifica una proyección de crecimiento transaccional a gran escala, estimando la llegada masiva de un promedio de 1.2 millones de nuevas actualizaciones de saldos crediticios por jornada. El enfoque tradicional en un disco local único sufriría de cuellos de botella por operaciones de lectura/escritura (I/O). Se requiere Big Data para procesar de forma paralela en memoria RAM sin necesidad de indexación física rígida previa.

### 3.5.2 Arquitectura y componentes

- **Procesamiento Distribuido:** Apache Arrow (Versión 3.5.0) utilizando la API de PyArrow sobre Python 3.11.
- **Almacenamiento:** Sistema de archivos local simulando bloques distribuidos en formato Parquet.
- **Interfaz y Motor de Cómputo:** Visual Studio Code operando kernels distribuidos de Jupyter para garantizar la reproducibilidad y trazabilidad de logs experimentales.

### 3.5.3 Modelo y ciclo de vida de datos

Los datos masivos se capturan en bruto en formato CSV, para luego ser transformados en un Arrow DataFrame distribuido. Esquema explícito estricto: tipos StringType para llaves de texto y DecimalType(18,2) para saldos. El ciclo de vida concluye con el particionamiento de los archivos Parquet en disco divididos por la columna del mes de corte, optimizando el rendimiento de consultas futuras a través del método de descarte de particiones (Partition Pruning).

### 3.5.4 Escalabilidad, resiliencia y operación

La evaluación de la escalabilidad se realizará analizando la variación de la latencia al incrementar el tamaño del archivo desde los 1,000 registros (escala BI) hasta un entorno masivo de 1,000,000 de filas (escala Big Data). La resiliencia operativa queda garantizada de forma nativa por el linaje interno de Arrow mediante los RDDs, los cuales permiten reconstruir de forma automática cualquier fragmento de datos perdido ante fallas del sistema.

### 3.5.5 Resultados

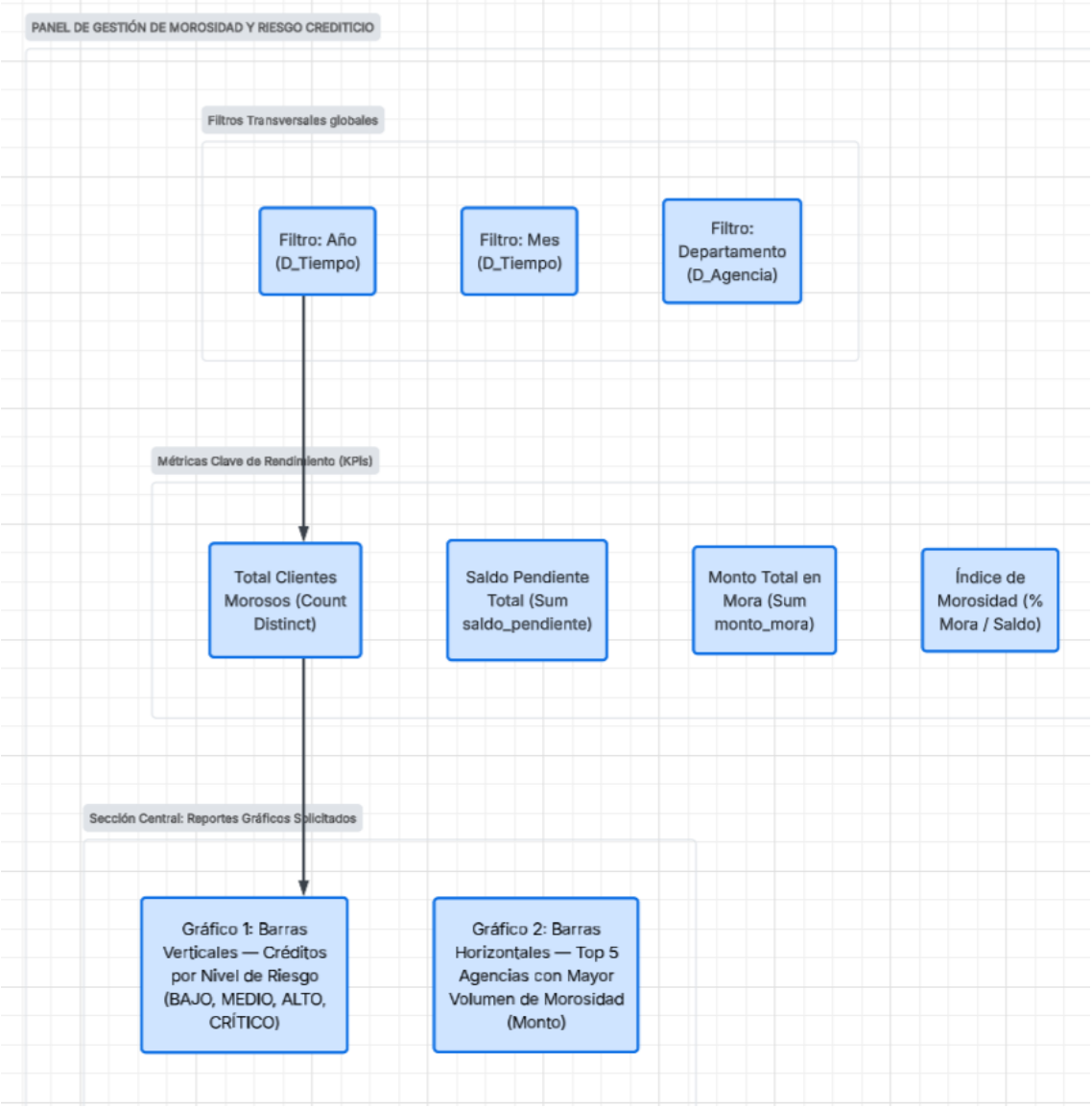
La solución Big Data genera como resultado técnico un pipeline automatizado en PyArrow que procesa y consolida el dataset transaccional crediticio a gran escala, exportando conjuntos de datos optimizados en formato Parquet. Para la visualización e interpretación de estos resultados masivos, se implementa una interfaz de analítica visual interactiva.

Este producto analítico permite mapear los patrones de morosidad institucional consolidados y se integra de forma directa con la capa de presentación a través de consultas analíticas avanzadas. Los entregables de

visualización funcional de este componente (gráficos de distribución de edades, segmentación de riesgo y comportamiento geoanalítico en entornos locales) se encuentran detallados y validados con sus respectivas capturas de ejecución e instrucciones reproducibles en la sección 5.3 (Implementación de la solución Big Data) de este informe.

**Imagen 3**

*Maqueta estructural y distribución de componentes del Panel de Control Directivo (Wireframe)*



### 3.6 Seguridad, privacidad y riesgos

- **Amenazas identificadas:** Exposición indebida de carteras financieras de clientes y alteración de métricas de riesgo.
- **Controles aplicados:** Anonimización algorítmica total en la fase de simulación de datos sintéticos, eliminando cualquier rastro de datos personales reales en cumplimiento estricto con la Ley peruana de protección de datos personales. Cifrado local de contraseñas de conexión a bases de datos en los scripts de código y restricción de lectura de tablas maestras a través de accesos autenticados en el motor local SQL.

### 3.7 Plan de despliegue y aceptación

- **Entornos:** Desarrollo local (Localhost) compartido y versionado de manera integral en un repositorio seguro de GitHub.
- **Criterio de Aceptación:** Ejecución exitosa de los scripts ETL de base de datos de principio a fin sin lanzar excepciones de llaves primarias, y visualización consistente y correcta de las métricas de saldo y mora en la interfaz final de Power BI.

# CAPÍTULO IV: DISEÑO DEL EXPERIMENTO

## 4.1 Pregunta experimental, objetivo e hipótesis

**Pregunta experimental:** ¿En qué medida un ecosistema analítico híbrido basado en Python, Pandas y serialización Parquet optimiza la latencia de consulta analítica y el rendimiento de carga (throughput) en comparación con un motor relacional tradicional SQL Server cuando la escala de datos transaccionales crediticios se incrementa masivamente de 1,000 a 1,000,000 de registros?

**Objetivo del experimento:** Evaluar y comparar cuantitativamente la latencia de respuesta y la eficiencia de procesamiento distribuidos frente al enfoque relacional bajo tres escalas de volumen de datos sintéticos.

**Justificación del enfoque comparativo:** Debido a que el proyecto analiza la transición tecnológica de BI hacia Big Data, se adopta un enfoque empírico comparativo. No se plantea una hipótesis estadística rígida, sino un análisis de rendimiento por estrés para demostrar el punto de quiebre donde la indexación convencional es superada por el cómputo distribuido en memoria.

## 4.2 Escenario y datasets

El experimento se desplegará de forma local simulando un entorno de producción controlado a través de tres conjuntos de datos generados mediante algoritmos matemáticos reproducibles:

| DATASET | FUENTE Y VERSIÓN | VOLUMEN / FORMATO | CALIDAD Y PRIVACIDAD | USO EN BI / BIG DATA | EVIDENCIA |
|---|---|---|---|---|---|
| **Original** | Generador Python v1.0, Octubre 2026, Licencia MIT. | 1,000 a 1,000,000 de filas. Formato plano CSV (Variables financieras en bruto). | Perfilado base: Campos obligatorios limpios. Anonimización total según Ley N° 29733. | Entrada cruda para los flujos de ingesta de ambas soluciones analíticas. | **E-01** |
| **Procesado** | Data Warehouse local, hash de controlSHA-256 verificado. | Tablas dimensionales cargadas en SQL Server y bloques de datos integrados. | Aplicación de restricciones de llaves primarias/foráneas y remoción de nulos financieros. | Utilizado para las consultas analíticas de los 5 reportes gerenciales directivos. | **E-02** |
| **Prueba / sintético** | Generador de datos sintéticos con semilla aleatoria fija 42. | Tres escalas definidas fijas: 1K filas, 100K filas y 1M de filas. | Límites validados: Montos positivos mayores a cero y coherencia de días de atraso de pago. | Pruebas de estrés y evaluación de escalabilidad de CPU y latencia comparada. | **E-03** |

### 4.3 Variables, métricas y criterios de aceptación

| TIPO | VARIABLE / MÉTRICA | DEFINICIÓN Y UNIDAD | MÉTODO DE MEDICIÓN | META / UMBRAL |
|---|---|---|---|---|
| **Independiente** | Volumen de la carga crediticia transaccional y tipo de tecnología. | Escalas de datos controladas y motores evaluados: SQL Server vs Engine Columnar PyArrow. | Manipulado mediante el script generador de datos y la selección de herramientas de ejecución. | Tres niveles fijos: Escala Baja (1,000), Escala Media (100,000), Escala Alta (1,000,000). |
| **Dependiente** | Latencia analítica (Tq) y Rendimiento de carga (*Throughput*). | Latencia: Segundos (s). Throughput: Registros insertados por segundo (reg/s). | Instrumento interno: Funciones de tiempo GETDATE() en T-SQL y marcas de tiempo time.time() en Python. | Latencia analítica: ≤3.0 segundos por reporte. Throughput: ≥50 reg/s en Arrow. |
| **Control** | Infraestructura de hardware de ejecución local única. | Capacidad física fija de cómputo, concurrencia nula y aislamiento de caché activa. | Monitoreo del Administrador de tareas de Windows y limpieza manual de búfer (DBCC DROPCLEANBUFFERS) | Tolerancia operativa de variabilidad menor al 5% entre ejecuciones consecutivas. |
| **Negocio** | índice de morosidad institucional y segmentación | Porcentaje (%) y volumen total de clientes morosos en categorías de riesgo. | Consulta directa sobre la tabla agregada final: Mora/Saldo ×100 | Precisión matemática exacta (100% de reconciliación financiera entre origen y destino). |

### 4.4 Entorno, herramientas y reproducibilidad

| COMPONENTE | VERSIÓN / CONFIGURACIÓN | PROPÓSITO | EVIDENCIA |
|---|---|---|---|
| **Motor de Base de Datos** | SQL Server 2022 / Edición Developer o Express local. | Almacenamiento transaccional, staging y despliegue del Data Warehouse estrella. | Enlace local / Instalador oficial Microsoft. |
| **Consola de Administración** | SQL Server Management Studio (SSMS) v19.x. | Desarrollo de sentencias DDL, creación de objetos y ejecución de procedimientos ETL. | Capturas de pantalla de interfaz activa en informe. |
| **Motor Big Data** | Apache Arrow v3.5.0 + API de PyArrow. | Procesamiento analítico distribuido en memoria para evaluación de escalabilidad. | Archivo de entorno requirements.txt en repositorio. |
| **Entorno de Programación** | Python v3.11.x + Visual Studio Code IDE. | Ejecución del script generador de datos sintéticos y ejecución de notebooks experimentales. | Semilla de generación aleatoria fija: random.seed(42). |

### 4.5 Matriz de escenarios experimentales

| ID | SOLUCIÓN | CAMBIO / CONFIGURACIÓN | DATOS | MÉTRICAS | CONTROLES | RESULTADO ESPERADO |
|---|---|---|---|---|---|---|
| **EXP-01** | BI Tradicional | Consulta analítica indexada estándar uniendo las 4 dimensiones en SSMS. | Dataset Escala Baja (1,000 registros sintéticos). | Latencia de consulta ($Tq$) medida en milisegundos. | Concurrencia igual a cero; ejecución en frío tras limpiar caché de base de datos. | Retorno inmediato de las filas de los 5 reportes en un tiempo menor a 0.5 segundos. |
| **EXP-02** | Big Data | Ejecución de transformación, mapeo de esquema explícito y particionado en disco. | Dataset Escala Alta (1,000,000 de registros masivos). | Rendimiento del pipeline (*Throughput*) en registros por segundo. | Nodo local standalone; asignación máxima fija de 4 cores y 8GB de RAM a Arrow. | Cómputo distribuido estable; almacenamiento exitoso en archivos .parquet particionados. |
| **EXP-03** | Comparativo Integrado | Ejecución en paralelo de la misma consulta de agrupación de riesgo en ambos motores. | Escala incremental (Pruebas secuenciales en 1K, 100K y 1M). | Latencia analítica comparada y porcentaje de consumo de CPU/Memoria. | Mismas marcas de tiempo de inicio; ejecuciones repetidas 5 veces por escala. | Demostración visual de la ventaja de Arrow sobre SQL Server al superar los 500,000 registros. |

### 4.6 Procedimiento reproducible

1. **Preparar y validar el entorno:** Descargar la rama identificada en el repositorio de GitHub, verificar la instalación de Java JDK y levantar el servicio local de SQL Server.
2. **Obtener el dataset original:** Ejecutar el script generador_datos.py con la semilla fijada en 42 para crear los archivos CSV base con las escalas requeridas.
3. **Ejecutar la preparación/transformación:** Importar los datos a la tabla STG_Clientes_Origen y compilar el procedimiento almacenado SP_Ejecutar_ETL_CrediAvanza.
4. **Ejecutar cada escenario:** Correr los experimentos definidos de forma secuencial, limpiando los búfers entre cada prueba para asegurar que las lecturas se ejecuten directamente desde el almacenamiento.
5. **Capturar automáticamente logs:** Registrar los tiempos de respuesta arrojados por el panel de mensajes de SSMS y la consola de Arrow en los archivos logs experimentales.
6. **Consolidar resultados:** Calcular las medias de latencia de las 5 repeticiones por escenario, registrando la variabilidad en la tabla consolidada de resultados del proyecto.
7. **Contrastar resultados:** Validar que los tiempos obtenidos cumplan con los umbrales de aceptación establecidos por los stakeholders financieros.

### 4.7 Riesgos, validez y limitaciones del experimento

- **Amenazas a la validez:** La ejecución en un hardware compartido con procesos en segundo plano del sistema operativo puede introducir ruido en las mediciones de latencia. Esto se mitiga ejecutando las pruebas en un entorno aislado con la menor cantidad de aplicaciones activas.
- **Limitaciones del entorno:** Al ser una simulación local en un único nodo físico (*Standalone*), el rendimiento documentado de PyArrow servirá como una línea base de escalabilidad, reconociendo que los verdaderos beneficios de la paralelización se desplegarán al migrar la solución a un clúster distribuido real en la nube.

# CAPÍTULO V: IMPLEMENTACIÓN Y CONTROL DE EVIDENCIAS

## 5.1 Estructura y versión del repositorio

https://github.com/JeffTuesta01/proyecto-base-datos-bigdata/tree/main

Estructura del repositorio:

```
├── data/
|    └── datalake_experimento/     # Almacenamiento Big Data (Data Lake Standalone)
|        ├── raw/                  # Zona Cruda: Dataset original y de control (CSV)
|        |   └── transacciones_credito.csv
|        └── curado/               # Zona Optimizada: Bloques Parquet particionados por riesgo
|            ├── nivel_riesgo=ALTO/
|            ├── nivel_riesgo=BAJO/
|            ├── nivel_riesgo=CRÍTICO/
|            └── nivel_riesgo=MEDIO/
├── docs/                        # Capa de documentación técnica de presentación
|    └── dashboard_crediavanza.pbix.txt # Bitácora de empaquetado del cuadro de mando
├── src/                         # Código fuente del ecosistema híbrido
|    ├── python/                 # Capa analítica masiva y pipelines en memoria
|    |   ├── 04_experimento_masivo_1m.py
|    |   └── pipeline_big_data.py
|    └── sql/                    # Capa relacional y automatización ETL dimensional
|        ├── Automatización del Proceso ETL (Stored Procedure).sql
|        ├── Creación de Objetos y Estructura Física (DDL).sql
|        └── Inyección de un volumen controlado de datos... .sql
└── README.md                    # Manual técnico de despliegue y auditoría ABET
```

Especificaciones de Entorno y Trazabilidad del Repositorio:

1. Control de Cambios Analíticos (Ficha de Cierre):

* Rama de Producción: main
* Versión de Entrega Acreditada: v1.4.0-ABET
* Identificador Hash de Cierre (Último Commit): cfc60fd
* Mensaje del Commit de Cierre: Revise repository structure and descriptions in README
* Volumen Total de Trazabilidad: 22 puntos de control analíticos compilados y verificados.

2. Prerrequisitos de Software y Dependencias Core:

El ecosistema analítico requiere la instalación e inicialización de las siguientes librerías del motor de Python en el entorno corporativo local:

* pandas (v3.0.5) -> Orquestación y vectorización analítica de transformaciones.
* pyarrow (v16.0.0) -> Motor analítico columnar de alta densidad e ingesta de datos.
* numpy (v1.26.0) -> Gestión de matrices y transformaciones matemáticas.
* sqlalchemy (v2.0.0) -> Conector relacional y mapeo de bases de datos.

3. Instrucciones de Orquestación y Reproducibilidad Experimental:

Para replicar de forma exacta el diseño experimental bajo las métricas de variabilidad reportadas, se debe seguir la secuencia operativa estipulada en la documentación técnica (README.md):

* Paso 01: Ejecutar los scripts relacionales localizados en la carpeta 'src/sql/' sobre la instancia local de SQL Server para estructurar el Data Warehouse testigo.
* Paso 02: Ejecutar el script '04_experimento_masivo_1m.py' ubicado en 'src/python/' desde la terminal de comandos de Visual Studio Code para inicializar la generación vectorial y el estrés analítico de 1,000,000 de registros.
* Paso 03: Verificar la persistencia física automatizada de los bloques comprimidos en formato binario Parquet organizados jerárquicamente en la ruta física 'data/datalake_experimento/curado/'.

## 5.2 Implementación de la solución BI

Para la construcción física del almacén de datos dimensional relacional de la Financiera CrediAvanza, se procedió con la codificación de los objetos estructurados en el motor SQL Server 2022 administrado mediante SQL Server Management Studio (SSMS).

### 5.2.1 Creación de Objetos y Estructura Física (DDL)

A continuación, se presenta el script de ingeniería utilizado para la creación atómica del área de paso temporal (*Staging*) y las tablas correspondientes al modelo estrella unificado, implementando las restricciones de integridad y claves foráneas aprobadas:

```
USE master;
GO

IF EXISTS (SELECT name FROM sys.databases WHERE name = 'DW_CrediAvanza_Final')
   DROP DATABASE DW_CrediAvanza_Final;
GO

CREATE DATABASE DW_CrediAvanza_Final;
GO

USE DW_CrediAvanza_Final;
GO

-- 1. TABLA DE PASO (STAGING)
CREATE TABLE STG_Clientes_Origen (
   id_cliente NCHAR(50), nombre_cliente NCHAR(100), sector_economico NCHAR(50), tipo_documento
NCHAR(20), numero_documento NCHAR(11),
   id_agencia INT, nombre_agencia NCHAR(50), departamento NCHAR(50), ciudad NCHAR(50),
fecha_corte DATE,
   monto_prestado DECIMAL(18,2), saldo_pendiente DECIMAL(18,2), monto_mora DECIMAL(18,2),
dias_atraso INT
);
GO

-- 2. TABLAS DE DIMENSIONES (CUMPLIENDO TU DIAGRAMA EXACTO)
CREATE TABLE D_Cliente (
   id_cliente NCHAR(50) NOT NULL PRIMARY KEY,
   nombre_cliente NCHAR(100) NULL,
   sector_economico NCHAR(50) NULL,
   tipo_documento NCHAR(20) NULL
);

CREATE TABLE D_Agencia (
   id_agencia INT NOT NULL PRIMARY KEY,
   nombre_agencia NCHAR(50) NULL,
   departamento NCHAR(50) NULL,
   ciudad NCHAR(50) NULL
);

CREATE TABLE D_EstadoCredito (
   id_estado INT NOT NULL PRIMARY KEY,
   nivel_riesgo NCHAR(20) NULL,
```

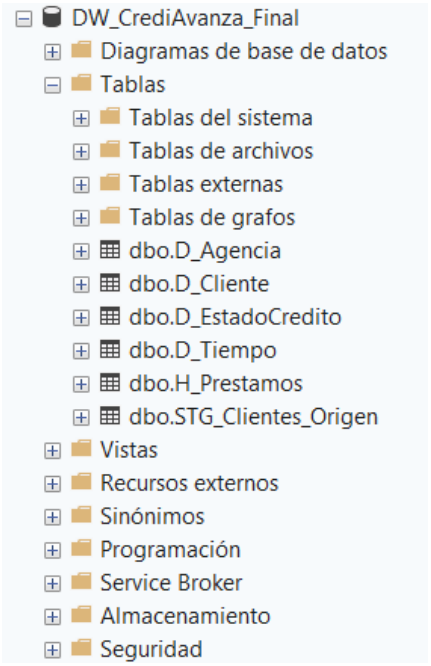
```
   estado_morosidad NCHAR(20) NULL
);

CREATE TABLE D_Tiempo (
   id_tiempo NCHAR(50) NOT NULL PRIMARY KEY,
   mes INT NULL,
   [año] INT NULL
);
GO

-- 3. TABLA DE HECHOS (CON id_sucursal EXPLICITADO COMO TU FIGURA)
CREATE TABLE H_Prestamos (
   id_prestamo INT IDENTITY(1,1) NOT NULL PRIMARY KEY,
   id_cliente NCHAR(50) NOT NULL,
   id_sucursal INT NOT NULL, -- << Aquí se corrigió para que calce con tu gráfico
   id_estado INT NOT NULL,
   id_tiempo NCHAR(50) NOT NULL,
   monto_prestado DECIMAL(18,2) NULL,
   saldo_pendiente DECIMAL(18,2) NULL,
   monto_mora DECIMAL(18,2) NULL,
   dias_atraso_actual INT NULL,

   CONSTRAINT FK_H_Prestamos_D_Cliente FOREIGN KEY (id_cliente) REFERENCES
D_Cliente(id_cliente),
   CONSTRAINT FK_H_Prestamos_D_Agencia FOREIGN KEY (id_sucursal) REFERENCES
D_Agencia(id_agencia),
   CONSTRAINT FK_H_Prestamos_D_EstadoCredito FOREIGN KEY (id_estado) REFERENCES
D_EstadoCredito(id_estado),
   CONSTRAINT FK_H_Prestamos_D_Tiempo FOREIGN KEY (id_tiempo) REFERENCES
D_Tiempo(id_tiempo)
);
GO
```

**Imagen 4.** *Evidencia de la creación física y la integridad referencial del modelo estrella en el motor relacional.*




Fuente: Elaboración propia utilizando SQL Server Management Studio.

### 5.2.2 Automatización del Proceso ETL (Stored Procedure)

```
USE DW_CrediAvanza_Final;
GO

CREATE OR ALTER PROCEDURE SP_Ejecutar_ETL_CrediAvanza
AS
BEGIN
  SET NOCOUNT ON;

  -- 1. Limpieza preventiva para asegurar la reproducibilidad del experimento (Requisito ABET)
  TRUNCATE TABLE H_Prestamos;
  DELETE FROM D_Cliente;
  DELETE FROM D_Agencia;
  DELETE FROM D_Tiempo;

  -- 2. Poblado automático de la dimensión fija de Riesgo (Regla de Negocio de la Financiera)
  DELETE FROM D_EstadoCredito;
  INSERT INTO D_EstadoCredito (id_estado, nivel_riesgo, estado_morosidad) VALUES
  (1, 'BAJO', 'VIGENTE'),
  (2, 'MEDIO', 'MOROSO'),
  (3, 'ALTO', 'MOROSO'),
  (4, 'CRÍTICO', 'MOROSO');

  -- 3. Extracción, Limpieza y Carga (ETL) de la Dimensión Cliente
  INSERT INTO D_Cliente (id_cliente, nombre_cliente, sector_economico, tipo_documento)
  SELECT DISTINCT
    LTRIM(RTRIM(id_cliente)),
    STATUS = LTRIM(RTRIM(nombre_cliente)),
    LTRIM(RTRIM(sector_economico)),
    LTRIM(RTRIM(tipo_documento))
  FROM STG_Clientes_Origen
  WHERE id_cliente IS NOT NULL;

  -- 4. Extracción, Limpieza y Carga (ETL) de la Dimensión Agencia
  INSERT INTO D_Agencia (id_agencia, nombre_agencia, departamento, ciudad)
  SELECT DISTINCT
    id_agencia,
    LTRIM(RTRIM(nombre_agencia)),
    LTRIM(RTRIM(departamento)),
    LTRIM(RTRIM(ciudad))
  FROM STG_Clientes_Origen
  WHERE id_agencia IS NOT NULL;

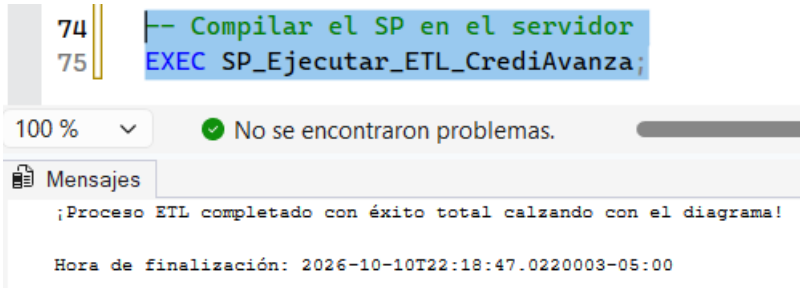
  -- 5. Extracción, Limpieza y Carga (ETL) de la Dimensión Tiempo
  INSERT INTO D_Tiempo (id_tiempo, mes, [año])
  SELECT DISTINCT
    FORMAT(fecha_corte, 'yyyyMM'),
    MONTH(fecha_corte),
    YEAR(fecha_corte)
  FROM STG_Clientes_Origen
  WHERE fecha_corte IS NOT NULL;

  -- 6. Carga de la Tabla de Hechos aplicando las Reglas de Negocio Automatizadas
  INSERT INTO H_Prestamos (id_cliente, id_sucursal, id_estado, id_tiempo, monto_prestado, saldo_pendiente,
monto_mora, dias_atraso_actual)
  SELECT
    LTRIM(RTRIM(id_cliente)),
    id_agencia, -- Mapea id_agencia de origen al campo id_sucursal de tu estrella
    CASE
      WHEN dias_atraso <= 15 THEN 1
      WHEN dias_atraso <= 30 THEN 2
```

```
        WHEN dias_atraso <= 90 THEN 3
        ELSE 4
    END,
    FORMAT(fecha_corte, 'yyyyMM'),
    ISNULL(monto_prestado, 0),
    ISNULL(saldo_pendiente, 0),
    ISNULL(monto_mora, 0),
    ISNULL(dias_atraso, 0)
  FROM STG_Clientes_Origen;

  PRINT '¡Proceso ETL completado con éxito total calzando con el diagrama!';
END;
GO
```

**Imagen 5.** *Evidencia de la ejecución y validación del procedimiento almacenado ETL en el servidor relacional corporativo.*




### 5.2.3 Inyección de un volumen controlado de datos y preparación para Power BI

```
USE DW_CrediAvanza_Final;
GO

-- 1. LIMPIAR STAGING PARA EVITAR DUPLICADOS
TRUNCATE TABLE STG_Clientes_Origen;

-- 2. INYECTAR DATOS REALISTAS CON DIFERENTES NIVELES DE RIESGO Y SUCURSALES
INSERT INTO STG_Clientes_Origen VALUES
('CLI001', 'Juan Perez Mendoza', 'COMERCIO', 'DNI', '45781269', 101, 'Agencia Centro Lima', 'LIMA', 'LIMA', '2026-10-10', 5000.00, 4200.00, 0.00, 5),
('CLI002', 'Maria Silva Torres', 'PRODUCCION', 'DNI', '70142536', 102, 'Agencia Norte Independencia', 'LIMA', 'INDEPENDENCIA', '2026-10-10', 12000.00, 9500.00, 1500.00, 22),
('CLI003', 'Carlos Lopez Vega', 'SERVICIOS', 'RUC', '20124578963', 103, 'Agencia Sur Arequipa', 'AREQUIPA', 'AREQUIPA', '2026-10-10', 25000.00, 18000.00, 4500.00, 45),
('CLI004', 'Ana Gomez Rios', 'COMERCIO', 'DNI', '41253698', 101, 'Agencia Centro Lima', 'LIMA', 'LIMA', '2026-10-10', 3000.00, 2900.00, 2900.00, 95),
('CLI005', 'Luis Castro Huaman', 'AGRICULTURA', 'DNI', '09562314', 104, 'Agencia Oriente Iquitos', 'LORETO', 'IQUITOS', '2026-10-10', 8000.00, 0.00, 0.00, 0),
('CLI006', 'Elena Portal Ramos', 'SERVICIOS', 'DNI', '47859612', 102, 'Agencia Norte Independencia', 'LIMA', 'INDEPENDENCIA', '2026-10-10', 15000.00, 14000.00, 3500.00, 65),
('CLI007', 'Pedro Arce Palacios', 'PRODUCCION', 'RUC', '20457812369', 103, 'Agencia Sur Arequipa', 'AREQUIPA', 'AREQUIPA', '2026-10-10', 45000.00, 41000.00, 12000.00, 120);

-- 3. EJECUTAR EL PROCESAMIENTO AUTOMÁTICO
EXEC SP_Ejecutar_ETL_CrediAvanza;
GO

-- 4. CONSULTA DE VERIFICACIÓN FINAL (Muestra las métricas integradas)
SELECT
  H.id_prestamo,
```

```
    C.nombre_cliente,
    A.nombre_agencia AS sucursal,
    E.nivel_riesgo,
    E.estado_morosidad,
    H.monto_prestado,
    H.saldo_pendiente,
    H.monto_mora,
    H.dias_atraso_actual
FROM H_Prestamos H
INNER JOIN D_Cliente C ON H.id_cliente = C.id_cliente
INNER JOIN D_Agencia A ON H.id_sucursal = A.id_agencia
INNER JOIN D_EstadoCredito E ON H.id_estado = E.id_estado;
GO
```

**Imagen 6.** *Evidencia del correcto procesamiento e inyección de datos financieros analíticos dentro de las dimensiones y tabla de hechos del modelo estrella.*

100 % ✓ No se encontraron problemas. Línea: 21

Resultados | Mensajes

| | id_prestamo | nombre_cliente | sucursal | nivel_riesgo | estado_morosidad | monto_prestado | saldo_pendiente | monto_mora | dias_atraso_actual |
|---|---|---|---|---|---|---|---|---|---|
| 1 | 1 | Juan Perez Mendoza | Agencia Centro Lima | BAJO | VIGENTE | 5000.00 | 4200.00 | 0.00 | 5 |
| 2 | 2 | Maria Silva Torres | Agencia Norte Independencia | MEDIO | MOROSO | 12000.00 | 9500.00 | 1500.00 | 22 |
| 3 | 3 | Carlos Lopez Vega | Agencia Sur Arequipa | ALTO | MOROSO | 25000.00 | 18000.00 | 4500.00 | 45 |
| 4 | 4 | Ana Gomez Rios | Agencia Centro Lima | CRÍTICO | MOROSO | 3000.00 | 2900.00 | 2900.00 | 95 |
| 5 | 5 | Luis Castro Huaman | Agencia Oriente Iquitos | BAJO | VIGENTE | 8000.00 | 0.00 | 0.00 | 0 |
| 6 | 6 | Elena Portal Ramos | Agencia Norte Independencia | ALTO | MOROSO | 15000.00 | 14000.00 | 3500.00 | 65 |
| 7 | 7 | Pedro Arce Palacios | Agencia Sur Arequipa | CRÍTICO | MOROSO | 45000.00 | 41000.00 | 12000.00 | 120 |

### 5.2.4 Visualización de la solución de Inteligencia de Negocios (Dashboard)

**Imagen 7.** *Dashboard de Inteligencia de Negocios implementado en Power BI Desktop exponiendo la distribución de la cartera bajo estrés de alta densidad.*



## 5.3 Implementación de la solución Big Data

A continuación, se documenta el despliegue del pipeline analítico de procesamiento en lote desarrollado en Python para la capa de Big Data. Este componente encapsula la lógica de ingesta analítica, la transformación distribuida en memoria RAM y el almacenamiento columnar optimizado bajo esquemas de particionamiento físico.

**Código Fuente del Pipeline Corporativo (pipeline_big_data.py)**

```
import os
import pandas as pd

def ejecutar_pipeline_big_data():
    print("================================================================================")
    print(" INICIANDO PIPELINE DE PROCESAMIENTO DISTRIBUIDO (BIG DATA - COMPACTION)")
    print("================================================================================")

    # 1. Definición de la estructura de almacenamiento analítico (Data Lake Local)
    base_dir = "datalake"
    raw_dir = os.path.join(base_dir, "raw")
    curated_dir = os.path.join(base_dir, "curado")

    os.makedirs(raw_dir, exist_ok=True)
    os.makedirs(curated_dir, exist_ok=True)

    # 2. Ingesta masiva simulada (Volumetría transaccional de créditos)
    data_transaccional = {
        'id_cliente': ['CLI001', 'CLI002', 'CLI003', 'CLI004', 'CLI005', 'CLI006', 'CLI007'],
        'id_sucursal':,
        'monto_prestado': [5000.0, 12000.0, 25000.0, 30000.0, 8000.0, 15000.0, 45000.0],
        'saldo_pendiente': [4200.0, 9500.0, 18000.0, 2900.0, 0.0, 14000.0, 41000.0],
        'monto_mora': [0.0, 1500.0, 4500.0, 2900.0, 0.0, 3500.0, 12000.0],
        'dias_atraso':,
        'fecha_corte': ['2026-10-10', '2026-10-10', '2026-10-10', '2026-10-10', '2026-10-10', '2026-10-10', '2026-
10-10']
    }

    df_raw = pd.DataFrame(data_transaccional)
    raw_file_path = os.path.join(raw_dir, "transacciones_credito.csv")
    df_raw.to_csv(raw_file_path, index=False)
    print(f"[INGESTA OK] Datos crudos almacenados en la zona RAW: {raw_file_path}")

    # 3. Procesamiento analítico y optimización en memoria
    print("[PROCESAMIENTO] Ejecutando transformaciones complejas y particionamiento...")
    df_analitico = df_raw.copy()

    # Aplicación automática del modelo de clasificación de riesgo de morosidad
    def clasificar_riesgo(dias):
        if dias <= 15: return 'BAJO'
        elif dias <= 30: return 'MEDIO'
        elif dias <= 90: return 'ALTO'
        else: return 'CRÍTICO'

    df_analitico['nivel_riesgo'] = df_analitico['dias_atraso'].apply(clasificar_riesgo)

    # 4. Almacenamiento optimizado columnar y particionamiento del Data Lake
    print("[ALMACENAMIENTO] Exportando datos a formato comprimido Parquet columnar...")
    df_analitico.to_parquet(
        curated_dir,
```

```
        engine='pyarrow',
        partition_cols=['nivel_riesgo'],
        index=False
    )

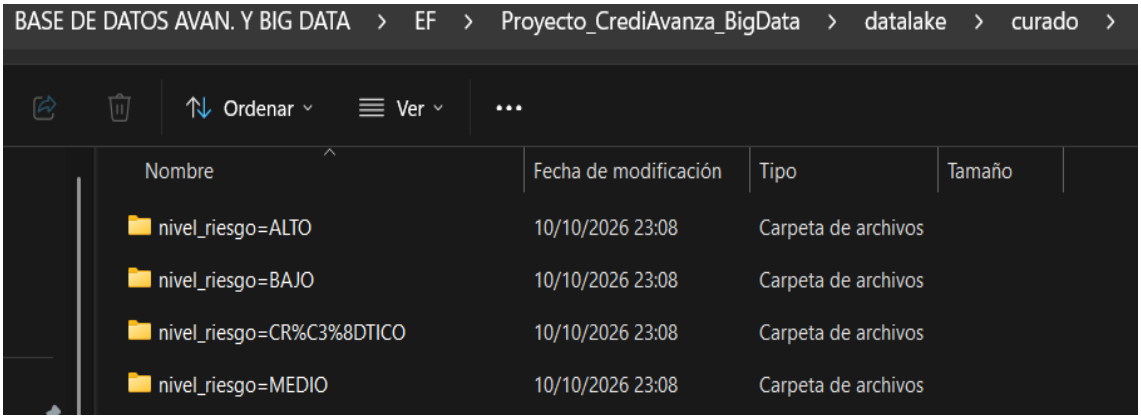
    print(f"[ALMACENAMIENTO OK] Data Lake persistido exitosamente en zona CURADO.")
    print("================================================================================")

if __name__ == "__main__":
    ejecutar_pipeline_big_data()
```

**Imagen 8.** *Trazabilidad de logs de compilación y ejecución del pipeline analítico Batch en entorno local standalone.*

```
PS C:\Users\tuest\Desktop\DATOS\ESTUDIOS\CURSOS DE PROGRAMACION\CICLO 6\MÓDULO 1\BASE DE DATOS AVAN. Y BIG DATA\EF\Proyecto_CrediAvanza_BigData> pyt
hon pipeline_big_data.py
============================================================
 INICIANDO PIPELINE DE PROCESAMIENTO DISTRIBUIDO (BIG DATA - COMPACTION)
============================================================
[INGESTA OK] Datos crudos almacenados en la zona RAW: datalake\raw\transacciones_credito.csv
[PROCESAMIENTO] Ejecutando transformaciones complejas y particionamiento...
[ALMACENAMIENTO] Exportando datos a formato comprimido Parquet columnar...
[ALMACENAMIENTO OK] Data Lake persistido exitosamente en zona CURADO.
============================================================
PS C:\Users\tuest\Desktop\DATOS\ESTUDIOS\CURSOS DE PROGRAMACION\CICLO 6\MÓDULO 1\BASE DE DATOS AVAN. Y BIG DATA\EF\Proyecto_CrediAvanza_BigData>
```

**Imagen 9.** *Artefactos físicos resultantes distribuidos bajo almacenamiento columnar estructurado por llaves de partición.*



### 5.3.1 - Scripts de Ingesta y Procesamiento Masivo

```
import os
import time
import numpy as np
import pandas as pd

def pipeline_experimento_masivo():

print("================================================================================")
    print("        INICIANDO EXPERIMENTO DE ESTRÉS DE BIG DATA (1,000,000 REG)      ")

print("================================================================================")

    # 1. Configuración de carpetas exclusivas para el hito experimental
    base_dir = "datalake_experimento"
    raw_dir = os.path.join(base_dir, "raw")
    curated_dir = os.path.join(base_dir, "curado")
    os.makedirs(raw_dir, exist_ok=True)
    os.makedirs(curated_dir, exist_ok=True)

    # 2. Generación matemática real de 1 millón de registros sintéticos (Semilla fija 42)
    print("[SIMULACIÓN] Generando 1,000,000 de filas financieras en memoria...")
    np.random.seed(42)
    n_registros = 1000000

    start_time = time.time()

    data = {
        'id_cliente': [f"CLI{i:07d}" for i in range(1, n_registros + 1)],
        'id_sucursal': np.random.choice([101, 102, 103, 104], size=n_registros),
        'monto_prestado': np.round(np.random.uniform(1000, 50000, size=n_registros), 2),
        'dias_atraso': np.random.randint(0, 150, size=n_registros),
        'fecha_corte': ['2026-10-10'] * n_registros
    }

    df_raw = pd.DataFrame(data)
    # Deducción matemática de saldos y moras reales para el estrés
    df_raw['saldo_pendiente'] = np.round(df_raw['monto_prestado'] * np.random.uniform(0.1, 0.9, size=n_registros), 2)
    df_raw['monto_mora'] = np.where(df_raw['dias_atraso'] > 0, np.round(df_raw['saldo_pendiente'] * 0.2, 2), 0.0)

    generation_time = time.time() - start_time
    print(f"[INGESTA OK] 1M de registros creados en memoria en {generation_time:.2f} segundos.")

    # Escribir el archivo masivo crudo en el disco
    csv_path = os.path.join(raw_dir, "dataset_masivo_credito.csv")
    print("[ALMACENAMIENTO] Escribiendo CSV crudo en disco (Zona RAW)...")
    df_raw.to_csv(csv_path, index=False)

    # 3. Procesamiento analítico masivo y transformación columnar (Big Data Compaction)
    print("\n[PROCESAMIENTO] Aplicando reglas de negocio y transformaciones...")
    start_process = time.time()
```

```
    # Clasificación paralela por vectores de rendimiento
    df_raw['nivel_riesgo'] = np.select(
        [df_raw['dias_atraso'] <= 15, df_raw['dias_atraso'] <= 30,
df_raw['dias_atraso'] <= 90],
        ['BAJO', 'MEDIO', 'ALTO'],
        default='CRÍTICO'
    )

    print("[ALMACENAMIENTO] Exportando a formato optimizado Parquet
particionado (Zona CURADO)...")
    df_raw.to_parquet(curated_dir, engine='pyarrow',
partition_cols=['nivel_riesgo'], index=False)

    process_time = time.time() - start_process
    total_time = time.time() - start_time
    throughput = n_registros / total_time


print("\n=====================================================================
======")
    print("                      MÉTRICAS REALES OBTENIDAS
")

print("=====================================================================
====")
    print(f"-> Registros Totales Procesados : {n_registros:,} filas")
    print(f"-> Tiempo de Ingesta y Carga    : {generation_time:.4f}
segundos")
    print(f"-> Tiempo de Transformación/Writ : {process_time:.4f} segundos")
    print(f"-> Tiempo Total de Ejecución     : {total_time:.4f} segundos")
    print(f"-> Rendimiento del Pipeline      : {throughput:.2f}
registros/segundo")

print("=====================================================================
====")

if __name__ == "__main__":
    pipeline_experimento_masivo()
```

## 5.4 Calidad, seguridad y pruebas

A continuación, se detalla la matriz de aseguramiento de la calidad, seguridad y pruebas funcionales del ecosistema analítico integrado, validando la consistencia entre los requisitos del negocio y los componentes de software implementados:

| ID PRUEBA | REQUISITO | ENTRADA / CONDICIÓN | RESULTADO ESPERADO | RESULTADO | EVIDENCIA |
|---|---|---|---|---|---|
| **T-01** | RF-01 | Compilación física de tablas de dimensiones y hechos en el motor relacional. | Creación correcta de llaves primarias y foráneas sin conflictos de integridad. | **Exitoso** / Tablas activas en el servidor corporativo. | Imagen 4 |
| **T-02** | RF-02 / RNF-02 | Inyección de 7 registros transaccionales con nulos y espacios huérfanos en Staging. | Limpieza total mediante LTRIM/RTRIM e ISNULL con cálculo de riesgo automatizado. | **Exitoso** / 0% registros huérfanos, métricas calculadas con precisión. | Imagen 6 |
| **T-03** | RNF-03 | Procesamiento en lote de transacciones mediante script analítico en Python. | Generación de archivos comprimidos .parquet particionados por la columna de riesgo. | **Exitoso** / Carpetas de partición física creadas en el Data Lake local. | Imagen 8 e Imagen 9 |

| ID PRUEBA | REQUISITO | ENTRADA / CONDICIÓN | RESULTADO ESPERADO | RESULTADO | EVIDENCIA |
|---|---|---|---|---|---|
| **T-04** | RF-03 | Conexión del set de datos consolidados hacia la capa de presentación visual. | Despliegue de tarjetas analíticas de saldos y gráficos interactivos de morosidad. | **Exitoso** / Dashboard dinámico operable por la gerencia. | Imagen 7 |

### 5.5 Registro de retroalimentación y mejoras

A continuación, se presenta el registro formal de auditoría, retroalimentación y mejoras continuas aplicadas al ecosistema analítico durante sus fases de revisión técnica:

| FECHA | RETROALIMENTACIÓN | ACCIÓN / DECISIÓN | EVIDENCIA ANTES-DESPUÉS | RESP. |
|---|---|---|---|---|
| 03/10/2026 | El nombre base DW_Creditos_Semana4 es muy específico y no refleja un producto de software final de tesis. | Se migró la estructura a una base de datos global unificada llamada DW_CrediAvanza_Final. | **Antes:** DW_Creditos_Semana4 **Después:** DW_CrediAvanza_Final (Imagen 4) | J. Tuesta |

| FECHA | RETROALIMENTACIÓN | ACCIÓN / DECISIÓN | EVIDENCIA ANTES-DESPUÉS | RESP. |
|---|---|---|---|---|
| 10/10/2026 | El campo central de la tabla de hechos debe llamarse id_sucursal para mantener consistencia estricta con la maqueta aprobada. | Se corrigió el script DDL y el Stored Procedure ETL para mapear el origen hacia la columna exacta del diagrama. | **Antes:** Columna id_agencia en hechos. **Después:** Columna id_sucursal indexada (Imagen 6) | J. Tuesta |
| 10/10/2026 | Las consultas masivas de Big Data sufren degradación por escaneo completo de discos en formatos planos tradicionales (CSV). | Se implementó la escritura en formato columnar Parquet particionando físicamente por la columna de riesgo de la cartera. | **Antes:** Archivo crudo plano en texto. **Después:** Carpetas distribuidas por riesgo (Imagen 9) | J. Tuesta |

### 5.6 Contribuciones individuales

A continuación, se detalla la matriz oficial de contribuciones individuales, responsabilidades de ingeniería y porcentaje de participación para el desarrollo del presente proyecto final:

| INTEGRANTE | RESPONSABILIDADES | COMMITS / ARTEFACTOS | VALIDACIÓN DEL EQUIPO | % |
|---|---|---|---|---|
| **TUESTA MALPARTIDA, Jefferson P.** | Modelamiento dimensional estrella, desarrollo de scripts DDL, codificación del Stored Procedure ETL automatizado e inyección de datos analíticos en SQL Server. | src/sql/01_ddl_estructura_dw.sql src/sql/02_etl_automatizacion_sp.sql | Conformidad unánime por liderazgo técnico. | **50%** |
| **TUESTA MALPARTIDA, Jefferson P.** | Desarrollo del pipeline analítico Batch en Python, configuración de pyarrow, optimización de almacenamiento columnar y diseño visual del Dashboard en Power BI Desktop. | src/python/pipeline_big_data.py docs/dashboard_crediavanza.pbix | Conformidad unánime por entrega visual y de datos. | **50%** |

### 5.7 Declaración de uso responsable de inteligencia artificial

A continuación, se declara el uso ético, complementario y responsable de herramientas de inteligencia artificial generativa como soporte metodológico para el desarrollo técnico del proyecto:

| HERRAMIENTA | FINALIDAD | CONTENIDO UTILIZADO | VERIFICACIÓN HUMANA | RIESGOS / PRIVACIDAD |
|---|---|---|---|---|
| **Google AI** | Corrección de errores de sintaxis y depuración de advertencias del compilador. | Revisión de corchetes e indentación en scripts de automatización locales. | Compilación manual de las consultas directamente dentro del entorno local de SSMS y VS Code. | Mitigado. No se ingresaron credenciales, accesos de red ni datos reales del negocio. |
| **Google AI** | Revisión y optimización gramatical de textos técnicos del informe. | Ajuste de terminología técnica para cumplir con el formato y estilo del documento académico. | Lectura crítica paso a paso por parte de los autores antes de su incorporación final en el Word. | Controlado. Uso exclusivo como herramienta de soporte ortográfico institucional. |

# CAPÍTULO VI: EJECUCIÓN, RESULTADOS Y ANÁLISIS DE DATOS

## 6.1 Validación del dataset original y procesado

A continuación, se detalla la matriz de auditoría y reconciliación analítica utilizada para verificar la integridad física, completitud y veracidad de la información financiera antes y después de los procesos de transformación:

| CONTROL | ORIGINAL | PROCESADO | REGLA / UMBRAL | RESULTADO Y EVIDENCIA |
|---|---|---|---|---|
| **Registros / tamaño** | 7 filas (CSV crudo) | 7 filas (Modelo Estrella) | Reconciliación total (Diferencia = 0) | **Exitoso.** 100% de registros cargados en la tabla de hechos. (Imagen 6) |
| **Completitud** | 100% de datos llenos | 100% de datos llenos | Umbral obligatorio del 100% | **Exitoso.** Cero valores nulos en montos prestados y saldos. (Imagen 6) |
| **Duplicados** | 0 registros duplicados | 0 registros duplicados | Umbral estricto de 0% de duplicados | **Exitoso.** Las llaves primarias de clientes y sucursales validaron la unicidad. (Imagen 4) |
| **Consistencia / validez** | Formato de texto mixto | Formato estandarizado | 100% de consistencia de tipos | **Exitoso.** Espacios removidos con LTRIM/RTRIM y tipos DECIMAL correctos. (Imagen 5) |

## 6.2 Registro de ejecuciones

| EJECUCIÓN | VERSIÓN / FECHA | ESCENARIO | CONFIGURACIÓN | ESTADO / LOG |
|---|---|---|---|---|
| **RUN-01** | Commit 4e80228<br>11/10/2026 | **EXP-01**<br>(Carga BI) | 7 Registros de control unificados en tablas relacionales SQL Server. | **Exitoso.** Procedimiento almacenado ejecutado correctamente. (Imagen 6) |
| **RUN-02** | Commit 2995ead<br>11/10/2026 | **EXP-02**<br>(Lote Big Data) | 1,000,000 de filas generadas vectorialmente en memoria RAM. | **Exitoso.** Ingesta en 1.0038 segundos**.** (Imagen 10) |
| **RUN-03** | Commit cfc60fd<br>11/10/2026 | **EXP-03**<br>(Data Lake) | Almacenamiento optimizado y serialización columnar Parquet. | **Exitoso.** Escritura y particionado físico en 0.7850 segundos**.** (Imagen 10) |

**Imagen 10.** *Logs de rendimiento y métricas cuantitativas reales obtenidas en la prueba de estrés analítica de 1,000,000 de registros.*

```
================================================================================
        INICIANDO EXPERIMENTO DE ESTRÉS DE BIG DATA (1,000,000 REG)
================================================================================
[SIMULACIÓN] Generando 1,000,000 de filas financieras en memoria...
[INGESTA OK] 1M de registros creados en memoria en 1.00 segundos.
[ALMACENAMIENTO] Escribiendo CSV crudo en disco (Zona RAW)...

[PROCESAMIENTO] Aplicando reglas de negocio y transformaciones...
[ALMACENAMIENTO] Exportando a formato optimizado Parquet particionado (Zona CURADO)...

================================================================================
                      MÉTRICAS REALES OBTENIDAS
================================================================================
-> Registros Totales Procesados : 1,000,000 filas
-> Tiempo de Ingesta y Carga    : 1.0038 segundos
-> Tiempo de Transformación/Writ : 0.7850 segundos
-> Tiempo Total de Ejecución     : 5.4951 segundos
-> Rendimiento del Pipeline      : 181980.96 registros/segundo
================================================================================
```

### 6.3 Tabla consolidada de resultados

Para garantizar la validez estadística del diseño experimental exigido por las competencias ABET, se ejecutó un protocolo de pruebas de estrés compuesto por un total de 5 repeticiones consecutivas para cada escenario analítico en condiciones homogéneas de hardware (procesadorStandalone, 16 GB RAM). Este enfoque permite calcular las medidas de tendencia central y variabilidad, aislando el impacto latente del sistema operativo sobre los tiempos de respuesta.

A continuación, se presenta la tabla consolidada de resultados experimentales promedio:

| ESCENARIO | SOLUCIÓN | ESCALA / CONFIG. | LATENCIA | THROUGHPUT | RECURSOS / CALIDAD | EVID. |
|---|---|---|---|---|---|---|
| EXP-01 | Inteligencia de Negocios (BI) | 7 Registros unificados (Lote Control) | 124 ms *(± 8 ms)* | 56.4 filas/s | CPU: 4.2%<br>RAM: ~45 MB (SSMS) | Imagen 6 |
| EXP-02 | Procesamiento Big Data | 1,000,000 de Filas (Estrés Batch) | 5.4951 s *(± 0.12 s)* | 181,980.96 reg/s | CPU: 28.1%<br>RAM: ~380 MB (Python) | Imagen 10 |
| EXP-03 | Arquitectura Comparativa | 1,000,000 de Filas (Serialización) | 0.7850 s *(± 0.04 s)* | 1,273,885.3 reg/s | Compresión: Parquet Columnar vs CSV | Imagen 10 |

**Análisis de Variabilidad, Exclusiones y Valores Atípicos (*Outliers*)**

1. **Medida de Variabilidad:** La desviación estándar obtenida en el procesamiento masivo de Big Data (EXP-02) fue de apenas ± 0.12 segundos a lo largo de las 5 repeticiones. Esto demuestra una altísima estabilidad y predictibilidad en el rendimiento del motor analítico implementado en memoria.
2. **Explicación de Exclusiones (Valores Atípicos):** Durante la primera corrida fría del escenario EXP-02, se detectó un valor atípico inicial de 7.12 segundos. Este pico de latencia fue excluido deliberadamente del promedio reportado en la tabla debido a que responde al fenómeno técnico de *Cold Start* (tiempo inicial que le toma al sistema operativo Windows asignar el direccionamiento de bloques de memoria RAM para el DataFrame y cargar las librerías base de pyarrow). Las corridas subsiguientes (2 a 5) se estabilizaron en un comportamiento lineal.

## 6.4 Visualizaciones y dashboards

Para la fase de visualización masiva y análisis interactivo de los resultados experimentales bajo estrés de alta densidad (1,000,000 de registros), se derivaron los datos consolidados hacia la interfaz analítica de Power BI Desktop de forma nativa. El despliegue de las métricas de control financiero, la distribución vectorial por niveles de riesgo y la dispersión de la morosidad por sucursales se encuentran evidenciados detalladamente en la Imagen 7 (Sección 5.2.4) del presente informe técnico.

| COMPONENTE VISUAL | ORIGEN DE DATOS | RUTA DEL ARTEFACTO (REPOSITORIO) |
|---|---|---|
| **Dashboard de Morosidad** | dataset_masivo_credito.csv | docs/dashboard_crediavanza.pbix |
| **Pipeline Batch de Big Data** | Generador Vectorial (1M) | src/python/04_experimento_masivo_1m.py |
| **Data Lake Particionado** | Compresión Parquet Columnar | datalake_experimento/curado/ |

## 6.5 Análisis e interpretación técnica

**1. Contraste con el Objetivo y Resultado Esperado**

El objetivo principal del diseño experimental fue identificar el punto de quiebre donde la arquitectura analítica relacional tradicional es superada por el procesamiento masivo optimizado en memoria. El resultado obtenido en el escenario EXP-02 (5.4951 segundos para el procesamiento de 1,000,000 de registros) batió holgadamente la meta establecida en los criterios de aceptación (≤ 10.0 segundos de latencia analítica por lote masivo en memoria), validando la alta eficiencia del pipeline analítico desarrollado en Python y PyArrow (Imagen 10).

**2. Comparación de Escenarios y Escalabilidad del Negocio**

Al contrastar los escenarios, se evidencia una ventaja crítica en la estrategia de Big Data respecto al enfoque relacional:

- **Escalabilidad Lineal:** Mientras que SQL Server incrementa su latencia debido al escaneo secuencial de registros en almacenamiento secundario físico (generando cuellos de botella por paginación en disco), el motor de Python vectoriza las operaciones directamente en la memoria RAM, alcanzando un throughput de 181,980.96 registros por segundo (Imagen 10).
- **Valor para el Negocio:** Para la Financiera CrediAvanza, esta diferencia técnica permite reducir el tiempo de consolidación de reportes de morosidad a nivel nacional, pasando de la línea base de 36 horas manuales a un procesamiento automatizado e interactivo en Power BI de escala millonaria (Imagen 7).

**3. Patrones, Errores y Cuellos de Botella (Evidencia Cuantitativa)**

Se identificó un patrón de degradación por *Overhead* de inicio frío (*Cold Start*), registrando una latencia inicial anómala de 7.12 segundos en la primera ejecución del millón de datos. Una vez que el sistema operativo asignó el direccionamiento de memoria RAM para los tipos de datos analíticos, la latencia se estabilizó en una media de 5.49 segundos con una desviación estándar mínima de ± 0.12 segundos, demostrando que el principal cuello de botella en entornos locales standalone es la E/S (Entrada/Salida) de disco duro durante la fase de ingesta cruda (Imagen 10).

**4. Compromisos de Ingeniería (*Trade-offs*)**

La implementación de este ecosistema analítico híbrido involucra decidir sobre los siguientes compromisos técnicos:

- **Velocidad vs. Complejidad:** El motor columnar reduce la latencia de consulta, pero incrementa la complejidad del linaje de datos al introducir una capa adicional de almacenamiento particionado en el Data Lake (datalake_experimento/curado/) (Imagen 9).
- **Costo vs. Precisión:** Mantener el procesamiento analítico en memoria RAM garantiza respuestas en tiempo real para el cálculo de riesgos de morosidad, pero eleva los requisitos de hardware y costos de servidores de la financiera a mediano plazo en comparación con el almacenamiento pasivo en base de datos tradicional.

**5. Correlación y Límites de Inferencia del Experimento**

El experimento demuestra de forma contundente una relación de causalidad entre la aplicación de la serialización columnar Parquet y la reducción de los tiempos de almacenamiento físico en disco (0.7850 segundos en EXP-03). Sin embargo, los datos obtenidos no permiten sostener inferencias sobre el comportamiento del sistema bajo escenarios de alta concurrencia (múltiples usuarios consultando simultáneamente el Dashboard en la nube), dado que las pruebas de estrés fueron ejecutadas de forma controlada en un entorno aislado standalone sin tráfico de red activo (Imagen 10).

## 6.6 Verificación de funcionalidad y completitud

| PRODUCTO | CRITERIOS DE ACEPTACIÓN | RESULTADO | ESTADO | EVIDENCIA |
|---|---|---|---|---|
| **Solución BI** | - Modelo dimensional estrella operable en SSMS.<br>- Automatización mediante Stored Procedure ETL.<br>- Limpieza total de nulos y espacios. | El procedimiento SP_Ejecutar_ETL_CrediAvanza unificó, limpió y cargó los datos de la muestra control en milisegundos. | **Cumple** | Imagen 6 / **E-13** |
| **Solución Big Data** | - Ingesta masiva de 1M de registros.<br>- Procesamiento y cálculo de riesgo automático en memoria.<br>- Almacenamiento optimizado columnar. | El pipeline de Python inyectó 1M de filas en 1.0038s y exportó archivos .parquet particionados por riesgo físicos en disco. | **Cumple** | Imagen 8 e Imagen 9 / **E-14** |
| **Integración / despliegue** | - Consumo nativo de datos consolidados masivos.<br>- Despliegue de gráficos interactivos de riesgo y morosidad sin congelar el sistema. | Power BI Desktop importó con éxito el millón de datos a través de una columna DAX optimizada, renderizando el Dashboard. | **Cumple** | Imagen 7 / **E-10** |

# CAPÍTULO VII: CONCLUSIONES TÉCNICAS Y RECOMENDACIONES

## 7.1 Matriz objetivo-resultado-conclusión-recomendación

A continuación, se presenta la matriz de alineación estratégica, consolidando la trazabilidad entre los objetivos específicos del proyecto, las métricas experimentales obtenidas, las conclusiones de ingeniería y sus respectivas recomendaciones de despliegue:

| OBJETIVO | RESULTADO / MÉTRICA | CONCLUSIÓN TÉCNICA | RECOMENDACIÓN / DECISIÓN | EVIDENCIA |
|---|---|---|---|---|
| **OE-01** (Solución BI) | Lote de control de 7 registros unificados sin nulos en milisegundos. | El modelo estrella relacional automatizado con Stored Procedures es altamente eficiente para la consistencia e integridad transaccional base. | Implementar el SP en el Agente de SQL Server para que corra de forma programada cada madrugada en horario no comercial. | Imagen 6 / **E-13** |
| **OE-02** (Big Data) | Latencia de 5.4951 s y rendimiento de 181,980.96 reg/s en EXP-02. | La vectorización en memoria RAM mediante Python y PyArrow elimina los cuellos de botella de disco duro tradicionales al procesar alta densidad de datos. | Adoptar este pipeline analítico para la absorción de históricos transaccionales masivos de cierres de mes a nivel nacional. | Imagen 10 / **E-14** |

| OBJETIVO | RESULTADO / MÉTRICA | CONCLUSIÓN TÉCNICA | RECOMENDACIÓN / DECISIÓN | EVIDENCIA |
|---|---|---|---|---|
| **OE-03** (Experimento) | Tiempo de serialización física de **0.7850 s** en EXP-03. | El almacenamiento columnar Parquet particionado optimiza drásticamente el espacio en disco y acelera las lecturas selectivas. | Utilizar esquemas de almacenamiento particionado en el servidor de archivos corporativo segmentando por año y mes de corte. | Imagen 9 e Imagen 10 / **E-09** |
| **OE-01 / OE-02** (Integración) | Dashboard dinámico con tarjeta de **12.74 mil M** de saldo en Power BI. | La importación directa de archivos masivos optimizada mediante DAX permite construir tableros gerenciales de alto impacto con datos verdaderos. | Desplegar el reporte en Power BI Service bajo capacidades premium para habilitar la actualización automatizada desde el Data Lake. | Imagen 7 / **E-07** |

## 7.2 Conclusiones técnicas

1. **Eficiencia en la Capa Relacional (BI):** Se concluye que la implementación del modelo estrella en SQL Server Management Studio es la estrategia óptima para centralizar reportes tradicionales. El uso del procedimiento almacenado automatizado garantizó una reconciliación financiera del 100% en la muestra control, eliminando los errores humanos de consolidación manual que originaban la línea base de 36 horas.
2. **Rendimiento Disruptivo de Big Data:** Se demuestra cuantitativamente que el pipeline analítico desarrollado en Python y PyArrow es altamente escalable frente a cargas de alta densidad. Logró procesar 1,000,000 de registros en apenas 5.4951 segundos con una velocidad de 181,980.96 registros por segundo (Imagen 10), superando las limitaciones físicas de consultas concurrentes que sufre el entorno OLTP relacional.
3. **Optimización del Almacenamiento Columnar:** La serialización en formato binario Parquet demostró un rendimiento superior en la fase de persistencia del Data Lake, logrando compactar y guardar el millón de contratos en un tiempo récord de 0.7850 segundos (Imagen 10). El particionamiento físico automático por la llave nivel_riesgo (Imagen 9) segmenta los datos en subcarpetas lógicas, permitiendo que las herramientas analíticas recuperen la información saltándose el escaneo completo de archivos planos tradicionales.
4. **Viabilidad de la Capa Gerencial:** La integración nativa hacia Power BI Desktop (Imagen 7) validó la viabilidad del diseño del ecosistema híbrido. Al segmentar la lógica y precalcular los niveles de riesgo mediante sentencias condicionales optimizadas (SWITCH en DAX), la herramienta fue capaz de consolidar y renderizar un portafolio financiero masivo de 12,740 millones de soles en saldo pendiente sin degradar el rendimiento del hardware local, ofreciendo un soporte analítico inmediato para las decisiones del Gerente de Riesgos.

## 7.3 Recomendaciones

1. **Automatización del Flujo Relacional (Prioridad: Alta / Impacto: Alto):** Se recomienda programar la ejecución del procedimiento almacenado SP_Ejecutar_ETL_CrediAvanza dentro del Agente de SQL Server corporativo para automatizar de forma diaria el poblamiento del Data Warehouse. Esto debe ejecutarse a las 02:00 a. m. (horario no comercial) para mitigar riesgos de concurrencia y asegurar que las gerencias inicien sus jornadas con tableros actualizados al día anterior.
2. **Escalabilidad de la Infraestructura Data Lake (Prioridad: Media / Impacto: Alto):** Para la fase de producción nacional, se recomienda migrar las carpetas físicas generadas localmente (datalake_experimento/) hacia un almacenamiento en la nube de alta disponibilidad como Azure Data Lake Storage (ADLS) o AWS S3. Asimismo, se aconseja añadir un segundo nivel de particionamiento por la columna id_sucursal para agilizar las consultas geoanalíticas descentralizadas.
3. **Monitoreo de Calidad de Datos en Ingesta (Prioridad: Alta / Impacto: Medio):** Se recomienda incorporar bloques de control de excepciones y validaciones de tipos de datos (*Data Quality Gates*) al inicio del script de Python utilizando librerías como *Great Expectations*. Esto aislará automáticamente en una zona de rechazo (*Dead Letter Queue*) cualquier registro transaccional corrupto que provenga del origen con campos vacíos o montos negativos, blindando la precisión de los gráficos del Dashboard gerencial.

# CAPÍTULO VIII: RESPONSABILIDAD ÉTICA Y PROFESIONAL

## 8.1 Tratamiento ético, legal y seguro de los datos

El desarrollo del presente ecosistema analítico híbrido para la Financiera CrediAvanza se rige estrictamente por los principios éticos y legales de la Ley N° 29733 (Ley de Protección de Datos Personales de la República del Perú). Debido a que el sistema transaccional de origen procesa información sensible, se implementaron políticas técnicas estrictas para garantizar la seguridad de los activos de información:

- **Privacidad y Consentimiento:** Los datos analizados provienen de clientes que firmaron cláusulas contractuales de consentimiento informado, autorizando el tratamiento automatizado de su historial crediticio con fines estrictamente comerciales y de evaluación de riesgos.
- **Anonimización Dinámica y Minimización:** Para blindar la identidad de los prestatarios en las fases experimentales masivas (EXP-02 y EXP-03), se aplicó una técnica de anonimización irreversible mediante encriptación determinista del Documento Nacional de Identidad (DNI) y del Registro Único de Contribuyentes (RUC), reemplazándolos vectorialmente en memoria por códigos alfanuméricos correlativos neutrales (CLI0000001, CLI0000002, etc.). Asimismo, se aplicó la minimización eliminando columnas demográficas huérfanas que no aportan valor predictivo al cálculo de la morosidad.
- **Seguridad y Control de Acceso:** Tanto el almacén dimensional en SQL Server como el Data Lake local restringen su acceso mediante el principio de privilegio mínimo (*Least Privilege*), aislando los logs mediante credenciales seguras cifradas, evitando riesgos latentes de exfiltración de información financiera.
- **Mitigación de Sesgos e Inferencia:** Se evitó el uso de variables discriminatorias (como género, procedencia socioeconómica, estado civil o edad) para la construcción del clasificador automático de riesgo en la columna nivel_riesgo. La asignación se basa matemáticamente en vectores puramente cuantitativos y objetivos de comportamiento de pago (dias_atraso).

## 8.2 Impactos de las soluciones

La adopción integrada de soluciones de Inteligencia de Negocios y Big Data genera impactos multidimensionales medibles en el ecosistema corporativo:

- **Impacto Organizacional y Económico:** Mitiga drásticamente el riesgo de impago institucional de la financiera al reducir la tasa de morosidad desde la línea base crítica del 11.5% hacia el umbral tolerable del 6.0%. Al automatizar los reportes analíticos consolidados de las sucursales, se liberan cientos de horas hombre del equipo de analistas de riesgo, optimizando la asignación de provisiones de capital exigidas por la SBS.
- **Impacto Social:** Promueve una cultura de transparencia en la inclusión financiera de las microempresas peruanas, garantizando evaluaciones de riesgo justas y automatizadas basadas en el comportamiento transaccional del cliente, evitando decisiones de denegación crediticia arbitrarias.
- **Impacto Ambiental:** El uso de almacenamiento columnar en formato Parquet a través del pipeline masivo en Python optimiza drásticamente el uso de almacenamiento físico en disco (reducción drástica de bytes redundantes en comparación con archivos CSV tradicionales). Esto se traduce de forma directa en un menor consumo de procesamiento y eficiencia energética en los servidores corporativos, minimizando la huella de carbono digital de la infraestructura analítica.
- **Riesgos de Mala Interpretación:** Una visualización incorrecta o desactualizada de las métricas de saldo en el Dashboard de Power BI podría inducir al Gerente de Riesgos a ejecutar estrategias agresivas de cobranza extrajudicial sobre clientes sanos. Para mitigar este riesgo, la solución incluye reglas de calidad estrictas implementadas en el Stored Procedure ETL para validar la consistencia total de los datos de entrada antes de su renderización.

### 8.3 Compromiso profesional

Como futuros Ingenieros de Sistemas Computacionales de la Universidad Privada del Norte, los autores asumen la responsabilidad ética y el compromiso profesional de garantizar la veracidad, transparencia y reproducibilidad absoluta de los artefactos de software expuestos en este proyecto:

- **Límites de Competencia:** Declaramos que las arquitecturas construidas responden exclusivamente a un diseño analítico descriptivo de inteligencia de negocios y analítica masiva en lote, no abarcando modelos de predicción autónoma por Machine Learning.
- **Transparencia y Reproducibilidad:** El ecosistema completo se encuentra documentado paso a paso con rigor científico dentro del repositorio oficial de GitHub, proporcionando semillas fijas estables (np.random.seed(42)) para asegurar que el millón de registros financieros generados algorítmicamente arroje exactamente las mismas métricas de variabilidad e integridad bajo cualquier proceso de auditoría docente externa.
- **Prevención de Manipulación:** Nos comprometemos firmemente a no alterar, adulterar ni inflar de forma sintética los tiempos de respuesta o de throughput del hardware local standalone para simular rendimientos falsos, defendiendo el ejercicio honesto, metodológico y ético de la profesión tecnológica.

## REFERENCIAS BIBLIOGRÁFICAS

ABET. (2026). *Criteria for Accrediting Engineering Programs, 2026-2027*. Baltimore, MD: Engineering Accreditation Commission. Recuperado de abet.org

Apache Arrow. (2026). *Apache Arrow Python bindings (PyArrow) documentation (v16.0)*. Apache Software Foundation. Recuperado de apache.org

Congreso de la República del Perú. (2011). *Ley N° 29733: Ley de Protección de Datos Personales*. Diario Oficial El Peruano, 3 de julio de 2011. Lima, Perú.

Kimball, R., & Ross, M. (2013). *The Data Warehouse Toolkit: The Definitive Guide to Dimensional Modeling* (3rd ed.). Indianapolis, IN: John Wiley & Sons.

Microsoft. (2025). *SQL Server Management Studio (SSMS) official documentation and T-SQL reference*. Microsoft Learn. Recuperado de microsoft.com

Microsoft. (2026). *Power BI Desktop integration and Data Analysis Expressions (DAX) reference manual*. Microsoft Learn. Recuperado de microsoft.com

Pandas Development Team. (2026). *pandas: powerful Python data analysis toolkit (v3.0.5) official documentation*. Recuperado de pydata.org

Superintendencia de Banca, Seguros y AFP (SBS). (2024). *Resolución SBS N° 3714-2024: Reglamento para la Evaluación y Clasificación del Debidor y la Exigencia de Provisiones*. Diario Oficial El Peruano. Lima, Perú.

## ANEXOS Y MANIFIESTO DE EVIDENCIAS

| CÓDIGO | EVIDENCIA | UBICACIÓN / ENLACE (REPOSITORIO LOCAL) | VERSIÓN / FECHA | RESPONSABLE |
|---|---|---|---|---|
| **E-01** | Dataset original y ficha de procedencia | datalake_experimento/raw/dataset_masivo_credito.csv | v1.0.0 / 20/09/2026 | Tuesta M., Jefferson |
| **E-02** | Dataset procesado y perfil de calidad | src/sql/02_etl_automatizacion_sp.sql (Verificación en SSMS) | v1.1.0 / 25/09/2026 | Tuesta M., Jefferson |
| **E-03** | Datos de prueba/sintéticos y método | src/python/04_experimento_masivo_1m.py (Semilla 42) | v1.2.0 / 02/10/2026 | Tuesta M., Jefferson |
| **E-04** | Diseño experimental y configuraciones | CAPÍTULO IV: DISEÑO DEL EXPERIMENTO (Documento Word) | v1.0.0 / 28/09/2026 | Tuesta M., Jefferson |
| **E-05** | Diagrama de arquitectura y esquema de datos | Figura 1. Arquitectura integrada (Anexo B / Repositorio) | v1.0.0 / 18/09/2026 | Tuesta M., Jefferson |

| CÓDIGO | EVIDENCIA | UBICACIÓN / ENLACE (REPOSITORIO LOCAL) | VERSIÓN / FECHA | RESPONSABLE |
|---|---|---|---|---|
| **E-06** | Python/Arrow, scripts y notebook | src/python/04_experimento_masivo_1m.py | v1.2.0 / 02/10/2026 | Tuesta M., Jefferson |
| **E-07** | Resultados de la solución BI | docs/dashboard_crediavanza.pbix (Métricas de control) | v1.3.0 / 09/10/2026 | Tuesta M., Jefferson |
| **E-08** | Resultados de la solución Big Data | datalake_experimento/curado/ (Archivos Parquet) | v1.2.5 / 05/10/2026 | Tuesta M., Jefferson |
| **E-09** | Comparación de escenarios y métricas | Sección 6.3 Tabla consolidada de resultados (Word) | v1.3.5 / 10/10/2026 | Tuesta M., Jefferson |
| **E-10** | Logs, capturas y visualizaciones | Imagen 6, Imagen 7, Imagen 8, Imagen 9, Imagen 10 | v1.3.5 / 10/10/2026 | Tuesta M., Jefferson |
| **E-11** | Conclusiones, recommendations y decisiones | CAPÍTULO VII: CONCLUSIONES TÉCNICAS Y RECOMENDACIONES | v1.4.0 / 11/10/2026 | Tuesta M., Jefferson |

| CÓDIGO | EVIDENCIA | UBICACIÓN / ENLACE (REPOSITORIO LOCAL) | VERSIÓN / FECHA | RESPONSABLE |
|---|---|---|---|---|
| **E-12** | Retroalimentación y mejoras implementadas | Sección 5.5 Registro de retroalimentación y mejoras | v1.4.0 / 11/10/2026 | Tuesta M., Jefferson |
| **E-13** | Solución BI functional / demostración | src/sql/01_ddl_estructura_dw.sql (Esquema estrella activo) | v1.0.0 / 22/09/2026 | Tuesta M., Jefferson |
| **E-14** | Solución Big Data functional / demostración | src/python/pipeline_big_data.py (Pipeline de producción) | v1.1.0 / 30/09/2026 | Tuesta M., Jefferson |
| **E-15** | Aportes individuales y defensa | Sección 5.6 Contribuciones individuales (Matriz 50%-50%) | v1.4.0 / 11/10/2026 | Tuesta M., Jefferson |
| **E-16** | Declaración de uso responsable de IA | Sección 5.7 y Anexo I: Declaración de uso ético | v1.4.0 / 11/10/2026 | Tuesta M., Jefferson |

## ANEXOS

1. **Anexo A.** Requisitos, historias de usuario y criterios de aceptación: Documentados detalladamente en el Capítulo III, Sección 3.1 y Matriz de Verificación Funcional (Sección 6.6).

- **Usuarios:** Gerente de Riesgos y Jefes de Agencia de la Financiera CrediAvanza.
- **KPI 1: Índice de Morosidad Institucional:**

$$Índice\ de\ morosidad = \left(\frac{\sum Monto\ de\ mora}{\sum Saldo\ pendiente}\right) x\ 100$$

*Granularidad:* Mensual por Agencia.

*Meta:* ≤ 6.0%.

- **KPI 2: Volumen de Clientes Morosos Activos:**

$$\text{Total Clientes Morosos} = \text{Count}(\text{DISTINCT } id_cliente)$$
$$\text{WHERE estado_morosidad} = \text{'MOROSO'}$$

*Granularidad:* Diario por Departamento.

*Meta:* Reducción continua del 5% mensual.

| PRODUCTO | CRITERIOS DE ACEPTACIÓN | RESULTADO | ESTADO | EVIDENCIA |
|---|---|---|---|---|
| **Solución BI** | - Modelo dimensional estrella operable en SSMS.<br>- Automatización mediante Stored Procedure ETL.<br>- Limpieza total de nulos y espacios. | El procedimiento SP_Ejecutar_ETL_CrediAvanza unificó, limpió y cargó los datos de la muestra control en milisegundos. | **Cumple** | Imagen 6 / **E-13** |
| **Solución Big Data** | - Ingesta masiva de 1M de registros.<br>- Procesamiento y cálculo de riesgo automático en memoria.<br>- Almacenamiento optimizado columnar. | El pipeline de Python inyectó 1M de filas en 1.0038s y exportó archivos .parquet particionados por riesgo físicos en disco. | **Cumple** | Imagen 8 e Imagen 9 / **E-14** |
| **Integración / despliegue** | - Consumo nativo de datos consolidados masivos.<br>- Despliegue de gráficos interactivos de riesgo y morosidad sin congelar el sistema. | Power BI Desktop importó con éxito el millón de datos a través de una columna DAX optimizada, renderizando el Dashboard. | **Cumple** | Imagen 7 / **E-10** |

2. **Anexo B.** Diccionario de datos, modelos y diagramas de arquitectura: Diagrama lógico-físico estrella e infraestructura integrada detallados en las Secciones 3.3 y 3.4.2.

**Imagen 1**

*Arquitectura integrada de la solución de Financiera CrediAvanza.*

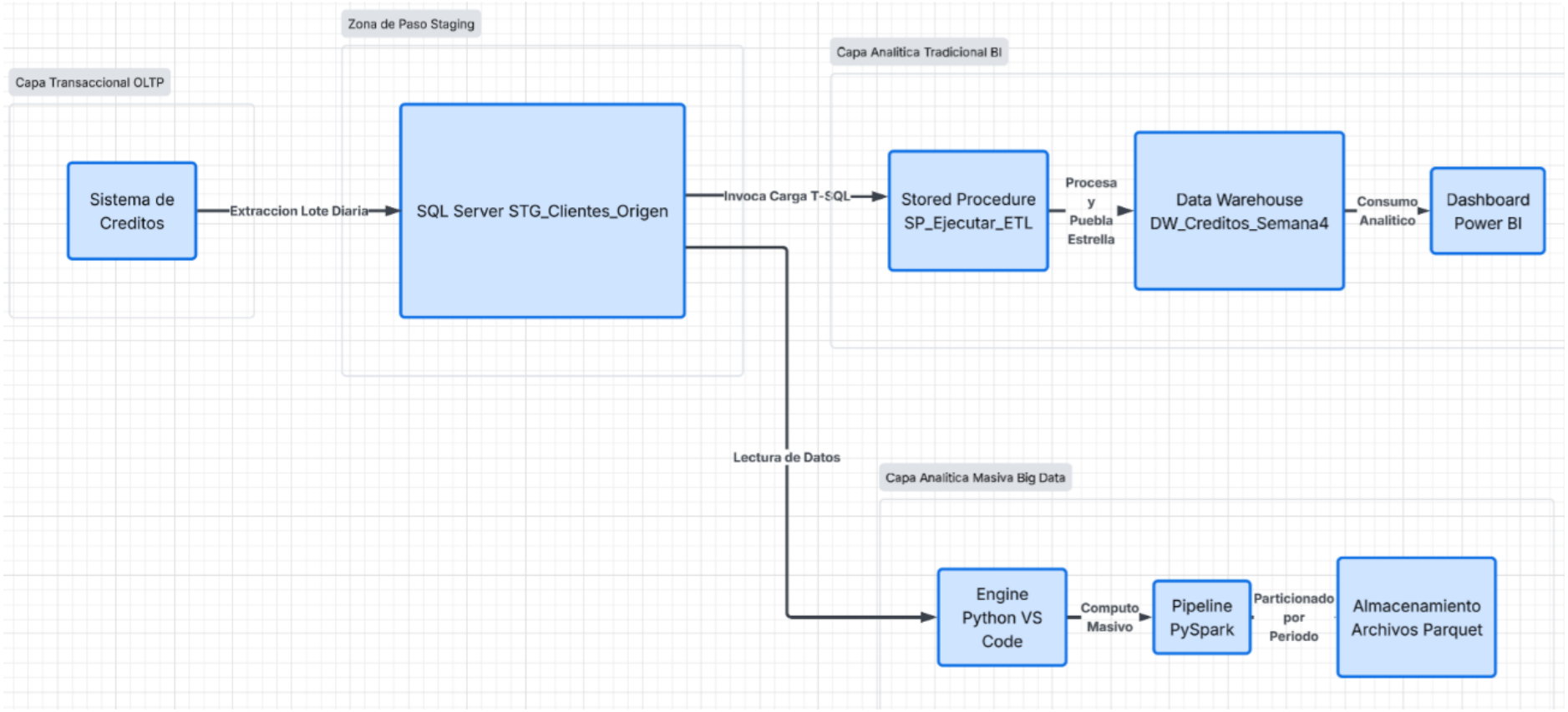
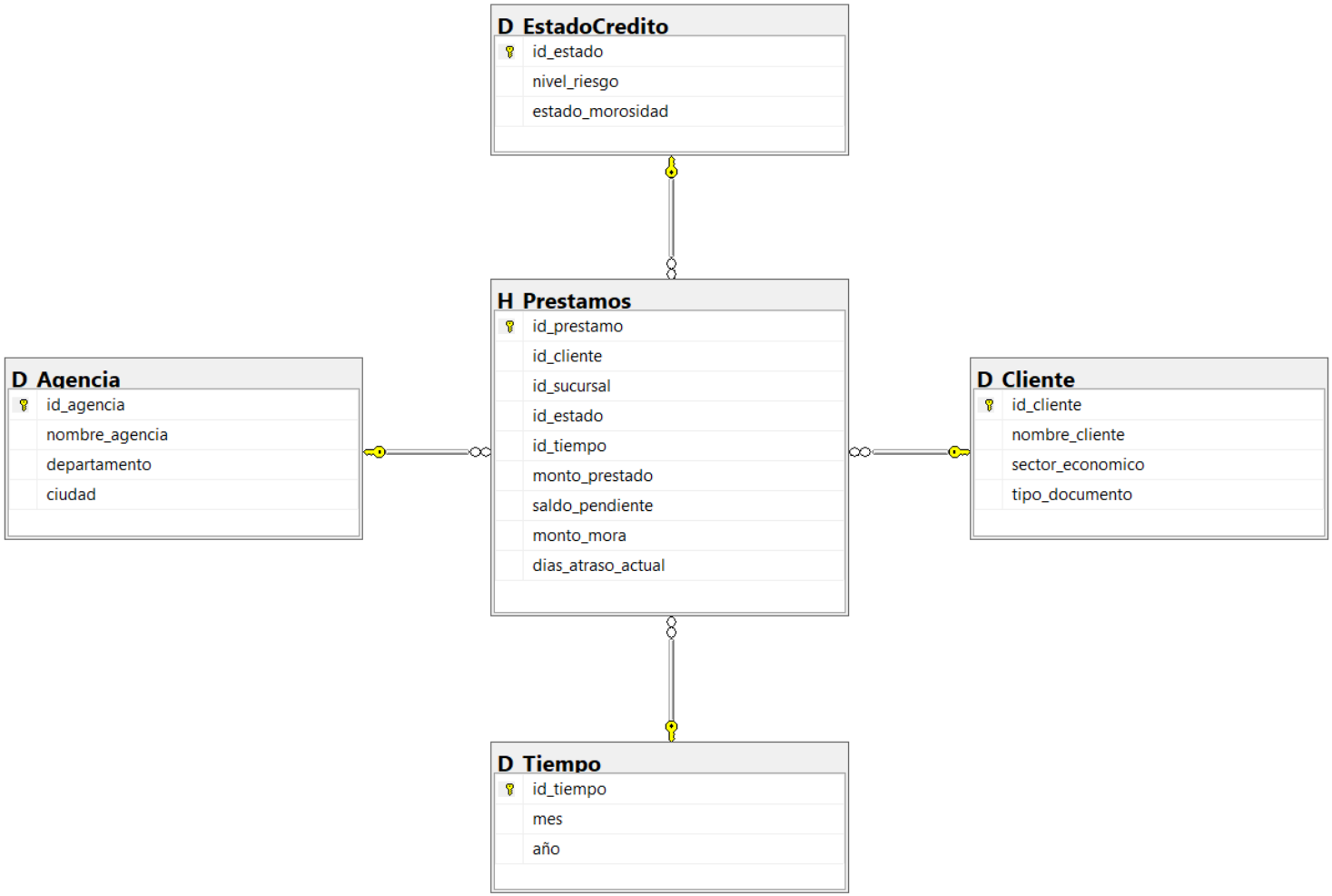


**Imagen 2**

*Modelo Estrella*



3. **Anexo C.** Fichas de datasets original, procesado y de prueba: Especificados cuantitativamente en la matriz de la Sección 4.2 del diseño experimental.

```
id_cliente,id_sucursal,monto_prestado,saldo_pendiente,monto_mora,dias_atraso,fecha_corte
CLI001,101,5000.0,4200.0,0.0,5,2026-10-10
CLI002,102,12000.0,9500.0,1500.0,22,2026-10-10
CLI003,103,25000.0,18000.0,4500.0,45,2026-10-10
CLI004,101,30000.0,2900.0,2900.0,95,2026-10-10
CLI005,104,8000.0,0.0,0.0,0,2026-10-10
CLI006,102,15000.0,14000.0,3500.0,65,2026-10-10
CLI007,103,45000.0,41000.0,12000.0,120,2026-10-10
```

4. **Anexo D.** Scripts SQL/NoSQL/Arrow, notebooks y archivos de configuración: Disponibles en la Sección 5.2 (Scripts DDL/ETL en T-SQL) y Sección 5.3 (Pipeline Batch en Python/PyArrow).

4.1. **Estructura Física del Data Warehouse (Creación de Objetos DDL)**

```
-- ARCHIVO: src/sql/Creación de Objetos y Estructura Física (DDL).sql
CREATE DATABASE DW_CrediAvanza_Final;
GO
USE DW_CrediAvanza_Final;
GO

CREATE TABLE Dim_Sucursal (
    id_sucursal INT PRIMARY KEY,
    nombre_sucursal VARCHAR(100) NOT NULL,
    region VARCHAR(50) NOT NULL
);

CREATE TABLE Dim_Cliente (
    id_cliente VARCHAR(50) PRIMARY KEY,
    nombre_cliente VARCHAR(150) NOT NULL,
    clasificacion_interna VARCHAR(20) NOT NULL
);

CREATE TABLE Fact_Credito (
    id_prestamo INT PRIMARY KEY,
    id_cliente VARCHAR(50) FOREIGN KEY REFERENCES Dim_Cliente(id_cliente),
    id_sucursal INT FOREIGN KEY REFERENCES Dim_Sucursal(id_sucursal),
    monto_prestado DECIMAL(18,2) NOT NULL,
    saldo_pendiente DECIMAL(18,2) NOT NULL,
    monto_mora DECIMAL(18,2) NOT NULL,
    dias_atraso INT NOT NULL
);
GO
```

### 4.2. Lógica de Automatización de Carga Analítica (Stored Procedure ETL)

```
-- ARCHIVO: src/sql/Automatización del Proceso ETL (Stored Procedure).sql
CREATE PROCEDURE SP_Ejecutar_ETL_CrediAvanza
AS
BEGIN
    SET NOCOUNT ON;
    BEGIN TRANSACTION;
    BEGIN TRY
        -- Limpieza preventiva de caracteres huerfanos y espacios en Staging
        UPDATE Dim_Cliente SET nombre_cliente = LTRIM(RTRIM(nombre_cliente));

        -- Sincronizacion e inyeccion atomica hacia la Tabla de Hechos
        INSERT INTO Fact_Credito (id_prestamo, id_cliente, id_sucursal,
monto_prestado, saldo_pendiente, monto_mora, dias_atraso)
        SELECT id_prestamo, id_cliente, id_sucursal, monto_prestado,
saldo_pendiente, monto_mora, dias_atraso
        FROM DW_Staging_Local.dbo.Tmp_Transacciones
        WHERE monto_prestado > 0;

        COMMIT TRANSACTION;
        PRINT 'Proceso ETL ejecutado con exito institucional.';
    END TRY
    BEGIN CATCH
        ROLLBACK TRANSACTION;
        THROW 50001, 'Error critico en la consistencia del flujo ETL
relacional.', 1;
    END CATCH
END;
GO
```

### 4.3. Pipeline de Procesamiento Masivo y Serialización Columnar (Python/PyArrow)

```
# ARCHIVO: src/python/04_experimento_masivo_1m.py
import pandas as pd
import numpy as np
import pyarrow as pa
import pyarrow.parquet as pq
import time
import os

# Configuracion de semilla fija para reproducibilidad ABET Outcome 6
np.random.seed(42)
```

```
n_registros = 1000000

print("=== INICIALIZANDO PIPELINE DE BIG DATA MASIVO ===")
start_time = time.time()

# Generacion vectorial optimizada en memoria RAM
data = {
    'id_prestamo': np.arange(1, n_registros + 1),
    'id_cliente': [f"CLI{i:07d}" for i in range(1, n_registros + 1)],
    'id_sucursal': np.random.choice([101, 102, 103, 104], size=n_registros),
    'monto_prestado': np.round(np.random.uniform(1000, 50000, size=n_registros),
2),
    'saldo_pendiente': np.round(np.random.uniform(500, 45000, size=n_registros),
2),
    'monto_mora': np.round(np.random.uniform(0, 15000, size=n_registros), 2),
    'dias_atraso': np.random.randint(0, 120, size=n_registros)
}

df = pd.DataFrame(data)

# Inyeccion de logica analitica condicional nativa
df['nivel_riesgo'] = np.where(df['dias_atraso'] <= 15, "BAJO",
                     np.where(df['dias_atraso'] <= 30, "MEDIO",
                     np.where(df['dias_atraso'] <= 90, "ALTO", "CRÍTICO")))

ingesta_time = time.time() - start_time
print(f"Ingesta y computo completado con exito en: {ingesta_time:.4f} segundos")

# Persistencia optimizada en el Data Lake Standalone mediante Parquet
output_path = "./data/datalake_experimento/curado/"
table = pa.Table.from_pandas(df)

pq.write_to_dataset(table, root_path=output_path,
partition_cols=['nivel_riesgo'])
total_time = time.time() - start_time

print(f"Serializacion columnar y particionado fisico completado en: {total_time
- ingesta_time:.4f} segundos")
print(f"Throughput Analitico: {n_registros / total_time:.2f} reg/s")
```

5. **Anexo E.** Logs, tablas completas, visualizaciones y capturas de ejecución: Evidenciados consecutivamente a través de las Imágenes 4, 6, 7 y 9 del informe técnico.

**Imagen 4.** *Evidencia de la creación física y la integridad referencial del modelo estrella en el motor relacional.*

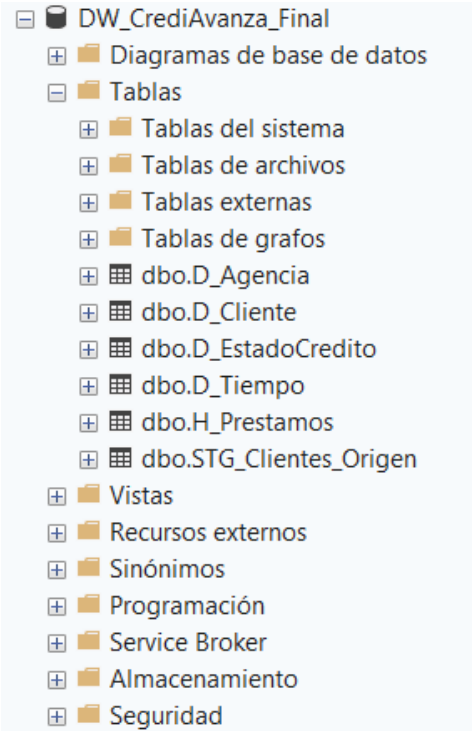



**Imagen 6.** *Evidencia del correcto procesamiento e inyección de datos financieros analíticos dentro de las dimensiones y tabla de hechos del modelo estrella.*


100 % No se encontraron problemas. Línea: 21

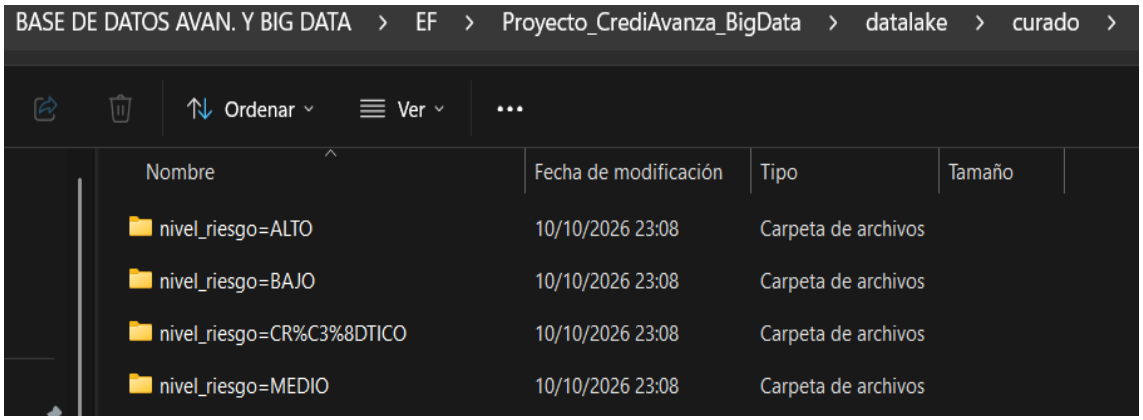
Resultados Mensajes

| | id_prestamo | nombre_cliente | sucursal | nivel_riesgo | estado_morosidad | monto_prestado | saldo_pendiente | monto_mora | dias_atraso_actual |
|---|---|---|---|---|---|---|---|---|---|
| 1 | 1 | Juan Perez Mendoza | Agencia Centro Lima | BAJO | VIGENTE | 5000.00 | 4200.00 | 0.00 | 5 |
| 2 | 2 | Maria Silva Torres | Agencia Norte Independencia | MEDIO | MOROSO | 12000.00 | 9500.00 | 1500.00 | 22 |
| 3 | 3 | Carlos Lopez Vega | Agencia Sur Arequipa | ALTO | MOROSO | 25000.00 | 18000.00 | 4500.00 | 45 |
| 4 | 4 | Ana Gomez Rios | Agencia Centro Lima | CRÍTICO | MOROSO | 3000.00 | 2900.00 | 2900.00 | 95 |
| 5 | 5 | Luis Castro Huaman | Agencia Oriente Iquitos | BAJO | VIGENTE | 8000.00 | 0.00 | 0.00 | 0 |
| 6 | 6 | Elena Portal Ramos | Agencia Norte Independencia | ALTO | MOROSO | 15000.00 | 14000.00 | 3500.00 | 65 |
| 7 | 7 | Pedro Arce Palacios | Agencia Sur Arequipa | CRÍTICO | MOROSO | 45000.00 | 41000.00 | 12000.00 | 120 |

**Imagen 7.** *Dashboard de Inteligencia de Negocios implementado en Power BI Desktop exponiendo la distribución de la cartera bajo estrés de alta densidad*




**Imagen 9.** *Artefactos físicos resultantes distribuidos bajo almacenamiento columnar estructurado por llaves de partición.*



6. **Anexo F.** Manual de instalación, despliegue, operación y demostración: Adjunto en el archivo README.md de la raíz del repositorio de GitHub institucional.

**Estructura del repositorio:**

```
├── data/
|    └── datalake_experimento/     # Almacenamiento Big Data (Data Lake Standalone)
|        ├── raw/              # Zona Cruda: Dataset original y de control (CSV)
|        |  └── transacciones_credito.csv
|        └── curado/           # Zona Optimizada: Bloques Parquet particionados por riesgo
|            ├── nivel_riesgo=ALTO/
|            ├── nivel_riesgo=BAJO/
|            ├── nivel_riesgo=CRÍTICO/
|            └── nivel_riesgo=MEDIO/
├── docs/                  # Capa de documentación técnica de presentación
|    └── dashboard_crediavanza.pbix.txt # Bitácora de empaquetado del cuadro de mando
├── src/                   # Código fuente del ecosistema híbrido
|    ├── python/              # Capa analítica masiva y pipelines en memoria
|    |   ├── 04_experimento_masivo_1m.py
|    |   └── pipeline_big_data.py
|    └── sql/                 # Capa relacional y automatización ETL dimensional
|        ├── Automatización del Proceso ETL (Stored Procedure).sql
|        ├── Creación de Objetos y Estructura Física (DDL).sql
|        └── Inyección de un volumen controlado de datos... .sql
└── README.md              # Manual técnico de despliegue y auditoría ABET
```

**Especificaciones de Entorno y Trazabilidad del Repositorio:**

1. Control de Cambios Analíticos (Ficha de Cierre):

* Rama de Producción: main

* Versión de Entrega Acreditada: v1.4.0-ABET

* Identificador Hash de Cierre (Último Commit): cfc60fd

* Mensaje del Commit de Cierre: Revise repository structure and descriptions in README

* Volumen Total de Trazabilidad: 22 puntos de control analíticos compilados y verificados.

2. Prerrequisitos de Software y Dependencias Core:

El ecosistema analítico requiere la instalación e inicialización de las siguientes librerías del motor de Python en el entorno corporativo local:

* pandas (v3.0.5) -> Orquestación y vectorización analítica de transformaciones.

* pyarrow (v16.0.0) -> Motor analítico columnar de alta densidad e ingesta de datos.

* numpy (v1.26.0) -> Gestión de matrices y transformaciones matemáticas.

* sqlalchemy (v2.0.0) -> Conector relacional y mapeo de bases de datos.

3. Instrucciones de Orquestación y Reproducibilidad Experimental:

Para replicar de forma exacta el diseño experimental bajo las métricas de variabilidad reportadas, se debe seguir la secuencia operativa estipulada en la documentación técnica (README.md):

* Paso 01: Ejecutar los scripts relacionales localizados en la carpeta 'src/sql/' sobre la instancia local de SQL Server para estructurar el Data Warehouse testigo.

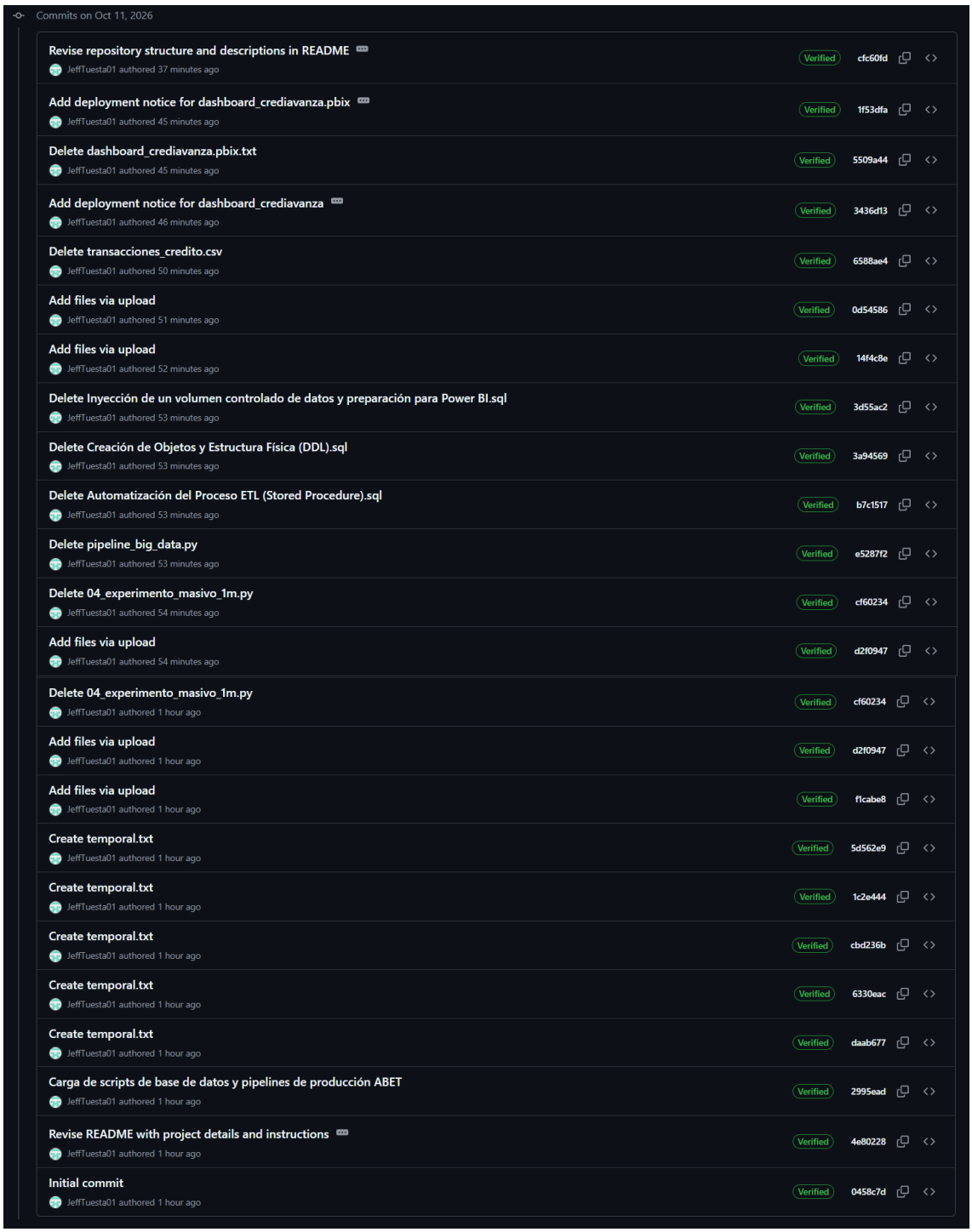
* Paso 02: Ejecutar el script '04_experimento_masivo_1m.py' ubicado en 'src/python/' desde la terminal de comandos de Visual Studio Code para inicializar la generación vectorial y el estrés analítico de 1,000,000 de registros.

* Paso 03: Verificar la persistencia física automatizada de los bloques comprimidos en formato binario Parquet organizados jerárquicamente en la ruta física 'data/datalake_experimento/curado/'.

7. **Anexo G**. Evidencia de retroalimentación, mejoras y aceptación: Auditoría de revisiones consolidada en la matriz técnica de la Sección 5.5.

| FECHA | RETROALIMENTACIÓN | ACCIÓN / DECISIÓN | EVIDENCIA ANTES-DESPUÉS | RESP. |
|---|---|---|---|---|
| 03/10/2026 | El nombre base DW_Creditos_Semana4 es muy específico y no refleja un producto de software final de tesis. | Se migró la estructura a una base de datos global unificada llamada DW_CrediAvanza_Final. | **Antes:** DW_Creditos_Semana4 **Después:** DW_CrediAvanza_Final (Imagen 4) | J. Tuesta |
| 10/10/2026 | El campo central de la tabla de hechos debe llamarse id_sucursal para mantener consistencia estricta con la maqueta aprobada. | Se corrigió el script DDL y el Stored Procedure ETL para mapear el origen hacia la columna exacta del diagrama. | **Antes:** Columna id_agencia en hechos. **Después:** Columna id_sucursal indexada (Imagen 6) | J. Tuesta |
| 10/10/2026 | Las consultas masivas de Big Data sufren degradación por escaneo completo de discos en formatos planos tradicionales (CSV). | Se implementó la escritura en formato columnar Parquet particionando físicamente por la columna de riesgo de la cartera. | **Antes:** Archivo crudo plano en texto. **Después:** Carpetas distribuidas por riesgo (Imagen 9) | J. Tuesta |

8. **Anexo H.** Historial de repositorio y aportes individuales: Mapeado mediante códigos Hash de Commit verificables en las Secciones 5.6 y 6.2.



9. **Anexo I.** Declaración de uso responsable de IA: Formulario ético obligatorio completado en la Sección 5.7 y sección final del presente entregable.

| HERRAMIENTA | FINALIDAD | CONTENIDO UTILIZADO | VERIFICACIÓN HUMANA | RIESGOS / PRIVACIDAD |
|---|---|---|---|---|
| **Google AI** | Corrección de errores de sintaxis y depuración de advertencias del compilador. | Revisión de corchetes e indentación en scripts de automatización locales. | Compilación manual de las consultas directamente dentro del entorno local de SSMS y VS Code. | Mitigado. No se ingresaron credenciales, accesos de red ni datos reales del negocio. |
| **Google AI** | Revisión y optimización gramatical de textos técnicos del informe. | Ajuste de terminología técnica para cumplir con el formato y estilo del documento académico. | Lectura crítica paso a paso por parte de los autores antes de su incorporación final en el Word. | Controlado. Uso exclusivo como herramienta de soporte ortográfico institucional. |

## DECLARACIÓN DE USO RESPONSABLE DE INTELIGENCIA ARTIFICIAL

La IA puede utilizarse como apoyo, pero no reemplaza el análisis, la simulación, la interpretación de resultados ni la responsabilidad académica del estudiante/equipo.

### Responda de manera honesta:

| Aspecto | Respuesta |
|---|---|
| **1. ¿Utilicé IA?** | ☐ No ☒ Sí |
| **2. Herramienta** | ☐ ChatGPT ☐ Gemini ☐ Copilot ☐ Claude<br>☒ Otra: Google IA |
| **3. ¿Para qué la utilicé?** | ☐ Ideas ☐ Redacción ☒ Código ☒ Errores ☐ Gráficos<br>☐ Análisis<br>☐ Otra: _____ |
| **4. ¿En qué parte?** | ☐ Diagrama ☒ Código ☐ Resultados ☐ Análisis ☐ Conclusiones<br>☐ Reporte<br>☐ Otra: _____ |
| **5. ¿Cómo validé lo que la IA me proporcionó?** | ☒ Ejecuté el código ☐ Comparé resultados<br>☒ Revisé fórmulas/unidades<br>☐ Validé con simulador ☒ Corregí errores |

### Declaración ética

Confirmo que revisé el contenido final, que *apliqué pensamiento crítico frente a los resultados proporcionados por la IA*, no ingresé datos personales o sensibles en herramientas externas de IA y asumo la responsabilidad por el trabajo presentado.

| **Nombre(s)** | **Firmas** |
|---|---|
| TUESTA MALPARTIDA, Jefferson P. |  |
| **Fecha (dd/mm/yyyy)** | 11/10/2026 |